Synthetic Synapses

System Documentation

Generated: March 22, 2026

Multi-Tenant HR Management System

Table of Contents

Overview

00 Documentation Structure -- NM i AI

Context

01 Architecture Overview

02 System State

03 Vision

Product

04 Product Requirements

Features

05 Autoloop -- Deep Technical Reference

06 Calibration Model -- Technical Reference

07 Daemon System -- Technical Reference

08 Exploration System -- Technical Reference

09 Astar Island Prediction System

10 GPU Simulator -- Deep Technical Reference

11 Multi-Researcher -- Technical Reference

12 Prediction Pipeline -- Step-by-Step Technical Reference

13 Astar Island - Technical Design

14 NorgesGruppen Object Detection

15 NorgesGruppen Detection - Technical Design

16 Tripletex AI Accounting Agent

Process

17 Development Workflow

18 Operations Guide

Documentation Structure -- NM i AI

Challenge documentation served via MCP at `https://mcp-docs.ainm.no/mcp`.

Challenge Documentation (`/docs`)

astar-island/

Challenge: Viking civilisation prediction (40x40x6 probability tensor)

- `overview.md`: Challenge description & rules
- `mechanics.md`: Simulator mechanics & cell types
- `endpoint.md`: API reference (`https://api.ainm.no/astar-island`)
- `scoring.md`: Entropy-weighted KL divergence scoring (0-100)
- `quickstart.md`: Getting started guide

norgesgruppen-data/

Challenge: Grocery product detection on store shelves

- `overview.md`: Challenge description & rules
- `submission.md`: Submission format, Docker sandbox, blocked imports
- `scoring.md`: 70% detection mAP + 30% classification mAP (mAP@0.5)
- `examples.md`: Example predictions & edge cases

tripletex/

Challenge: AI accounting agent via Tripletex API

- `overview.md`: Challenge description & rules
- `endpoint.md`: HTTPS `/solve` endpoint spec
- `scoring.md`: Correctness x tier multiplier + efficiency bonus
- `examples.md`: Example tasks & responses
- `sandbox.md`: Sandbox environment details

Solution Documentation (`/docs/0*`)

00-context/

WHY and WHAT EXISTS -- system-level understanding

- `vision.md` : Project purpose, challenge portfolio, design principles
- `system-state.md` : Running processes, data volumes, scores, external dependencies
- `architecture.md` : Repository layout, system diagrams, technology stack

01-product/

WHAT the product must do -- requirements

- `prd.md` : Per-challenge objectives, scoring, constraints, requirements checklist

02-features/

HOW features are designed & built

astar-island-prediction/

- `feature-spec.md` : System overview, subsystems, user flows, edge cases
- `tech-design.md` : Data models, API endpoints, pipeline diagram, file map
- `autoloop-deep-dive.md` : 44-parameter space, FastHarness pre-computation, evaluation pipeline, optimization loop, perturbation strategy, experiment log format
- `gpu-simulator.md` : GPUSimulator internals (batched CUDA), 17 simulator params, CMA-ES fitting (GT + observations), regime warm starts, transfer learning, adaptive alpha
- `prediction-pipeline.md` : 11-stage step-by-step walkthrough of `predict_gemini.py`
- `calibration-model.md` : Hierarchical prior (fine/coarse/base/global), feature key definition, training, inference, regime-conditional weighting
- `daemon-system.md` : Round monitor, submission pipeline, GPU resubmission loop, calibration download, error handling
- `exploration-system.md` : Grid/entropy strategies, data structures (ObservationAccumulator, GlobalMultipliers, FeatureKeyBuckets), overlay functions, terrain openness diffusion
- `multi-researcher.md` : LLM research agent (Gemini Pro+Flash), orchestration loop, idea evaluation, knowledge base, integration path

norgesgruppen-detection/

- `feature-spec.md` : Two-stage pipeline overview, training pipeline, acceptance criteria

- `tech-design.md` : Inference stages (YOLO -> WBF -> ConvNeXt-V2 -> blend), training architecture, sandbox compatibility, timing budget, file map

tripletex-agent/

- `feature-spec.md` : Challenge overview, scoring, task execution flow (not started)

04-process/

HOW to work with this system

- `operations.md` : Full system startup, training, submission, monitoring, troubleshooting
- `dev-workflow.md` : Daily development loops, pre-submission checklist, general rules

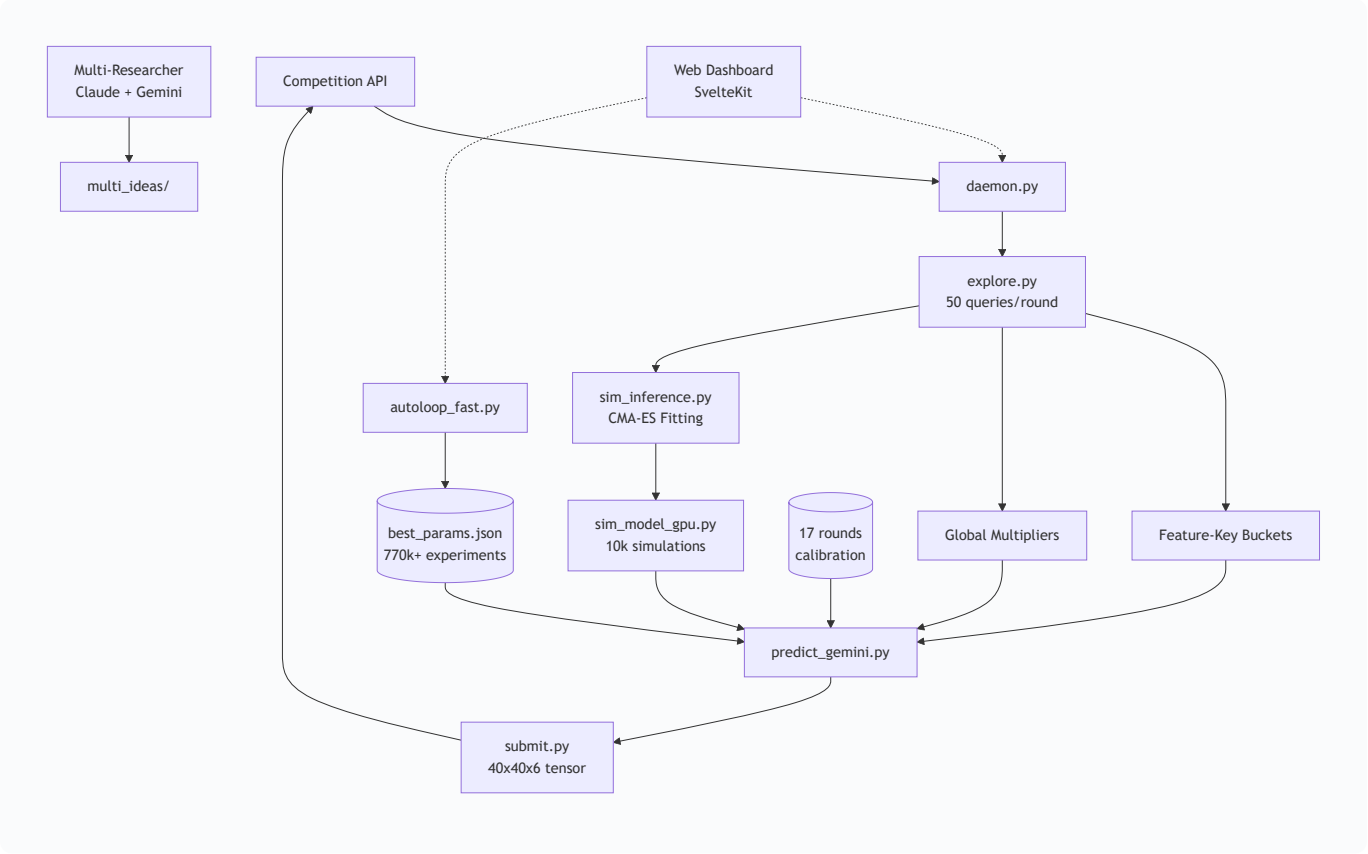
Architecture Overview

High-level architecture of all systems in the NM i AI project.

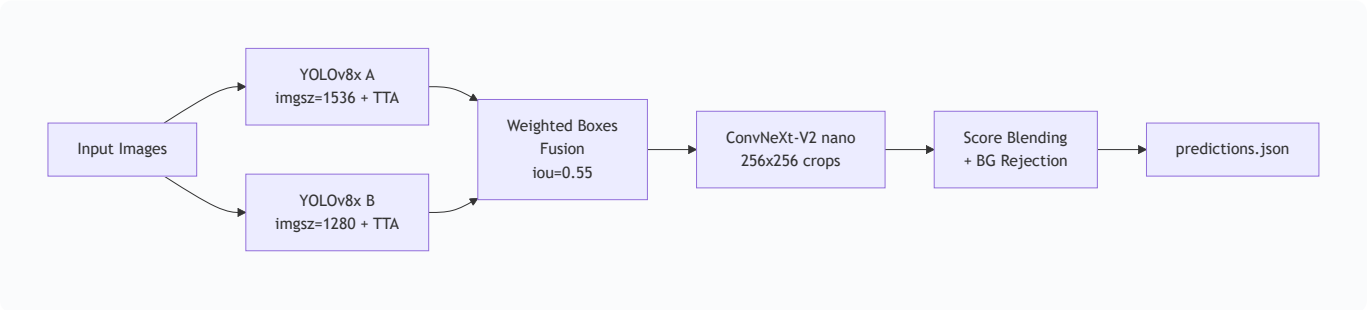
Repository Layout

```
AINMNO/
├── docs/                          # Challenge documentation & training data
│   ├── astar-island/              # Challenge spec, API reference, scoring
│   ├── norgesgruppen-data/        # COCO dataset (248 images, 357 categories)
│   └── tripletex/                 # Accounting agent spec
├── astar-island-solution/         # Viking prediction system (~50 Python files)
│   ├── web/                       # SvelteKit monitoring dashboard
│   └── data/                      # Calibration, caches, experiments, logs
├── norgesgruppen-solution/        # Object detection pipeline
│   ├── datasets/                  # YOLO-format training data
│   └── classifier_data/          # Classifier crops & references
└── submission_sub*/               # Archived NorgesGruppen submissions
```

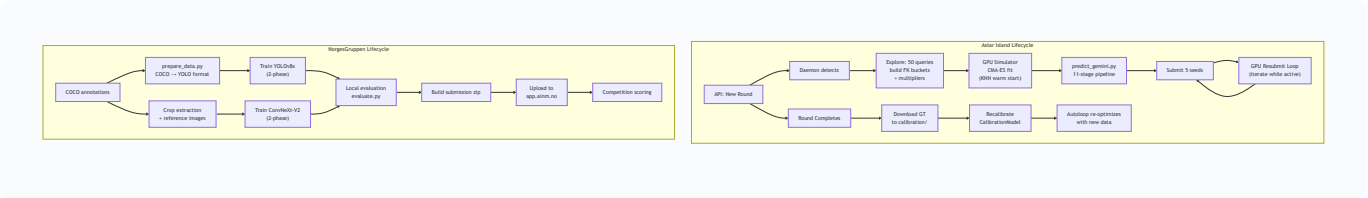
Astar Island System Architecture



NorgesGruppen Detection Pipeline



End-to-End Data Lifecycle



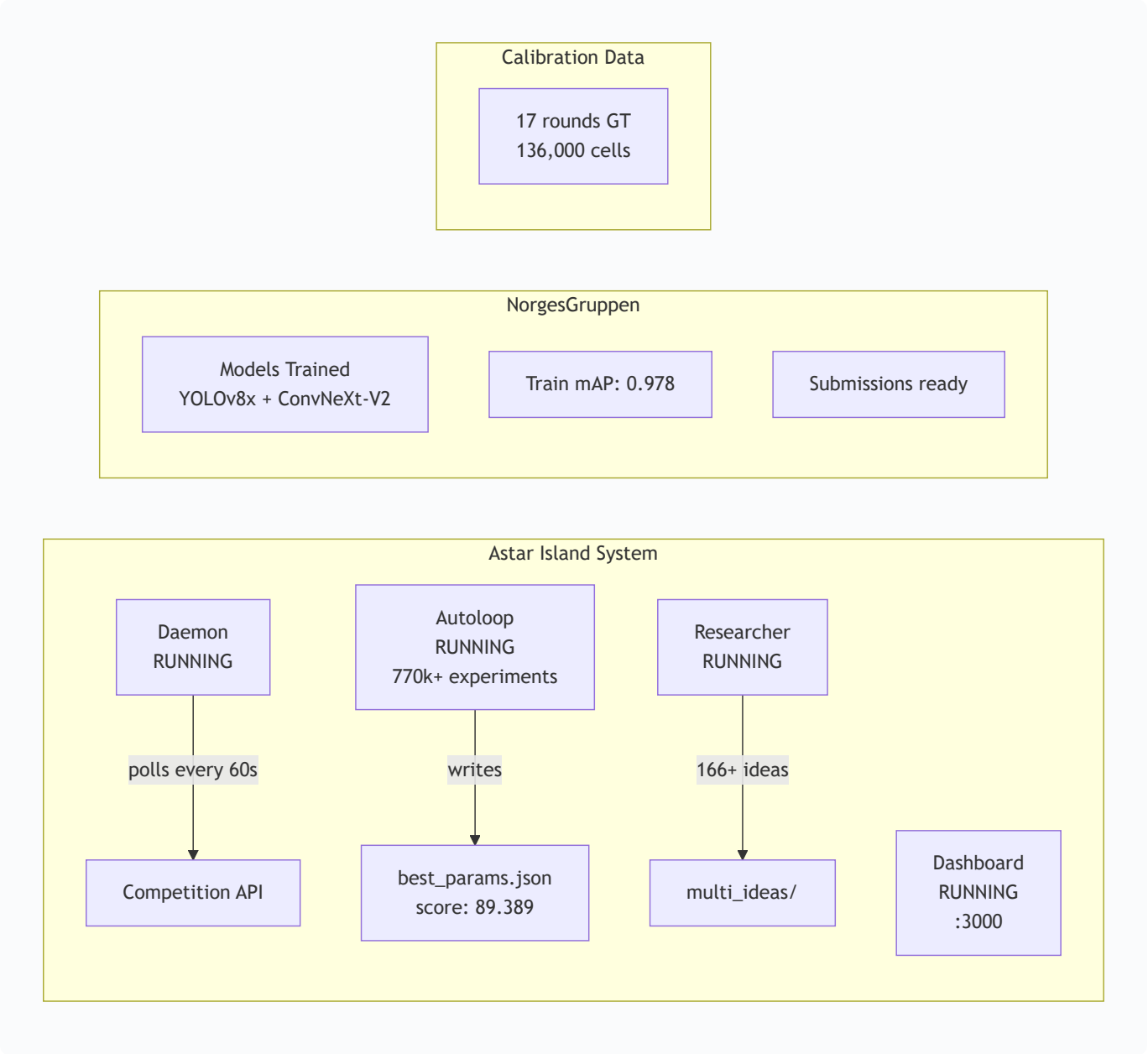
Technology Stack

Layer	Astar Island	NorgesGruppen
Language	Python 3.11	Python 3.11
ML Framework	NumPy, PyTorch (GPU sim)	PyTorch 2.6.0, ultralytics 8.1.0
Models	Parametric simulator, calibration	YOLOv8x, ConvNeXt-V2 nano
Optimization	CMA-ES, Metropolis-Hastings	Grid sweep, manual tuning
Frontend	SvelteKit + TypeScript	--
Serialization	JSON, JSONL	safetensors, .pt
Deployment	Local daemon + autoloop	Zip submission to Docker sandbox

System State

Current state of all running systems and infrastructure as of 2026-03-21.

System Health Overview



Running Processes

Process	Script	Purpose	Flag
Daemon	`astar-island-solution/daemon.py`	Monitors API for new rounds, auto-submits	`-u` (unbuffered)
Autoloop Fast	`astar-island-solution/autoloop_fast.py`	Continuous parameter optimization (~70ms/exp)	`-u`
Multi-Researcher	Various research agents	Structural algorithm improvements via LLMs	`-u`
Web Dashboard	`astar-island-solution/web/`	SvelteKit monitoring UI on localhost:3000	--

All Python processes require `-u` flag for real-time log output.

Data Volumes

Directory	Contents	Size
`astar-island-solution/data/calibration/`	17 rounds of ground truth	~50 MB
`astar-island-solution/data/multi_ideas/`	165+ generated algorithm variants	~15 MB
`astar-island-solution/data/sim_cache/`	Simulator state caches (15 rounds)	~10 MB
`norgesgruppen-solution/runs/`	YOLO training checkpoints	~2 GB
`norgesgruppen-solution/*.pt`	Trained model weights	~530 MB

Key Scores

Challenge	Metric	Current Best
Astar Island	Entropy-weighted KL (0-100)	89.4 avg (90.2 non-boom, 87.7 boom)
NorgesGruppen	70% det mAP + 30% cls mAP	0.978 (train set)

External Dependencies

Service	URL	Purpose
Competition API	`https://api.ainm.no/astar-island`	Astar Island simulation & submission
Competition App	`https://app.ainm.no`	Leaderboard, submission UI
MCP Docs Server	`https://mcp-docs.ainm.no/mcp`	Competition documentation

Vision

NM i AI (Norwegian AI Championship) is a competitive AI platform at app.ainm.no. This repository contains solutions for all four challenges, with active development on two.

Purpose

Build competition-grade AI systems across four distinct domains: computer vision, probabilistic prediction, agentic accounting, and game AI. The goal is to maximize leaderboard scores through algorithmic innovation, hyperparameter optimization, and autonomous experimentation infrastructure.

Boundaries

- **In scope:** All four challenges, shared infrastructure, autonomous optimization systems
- **Out of scope:** Production deployment, multi-user support, long-term maintenance

Challenge Portfolio

Challenge	Domain	Status	Approach
NorgesGruppen Data	Object Detection	Active, submitted	YOLOv8x + ConvNeXt-V2 ensemble
Astar Island	Probabilistic Prediction	Active, autonomous	Bayesian calibration + simulator + autoloop
Tripletex	AI Accounting Agent	Not started	LLM agent + API integration
Grocery Bot	Game AI	Skipped	Not scored, warm-up only

Key Design Principles

1. **Autonomous optimization over manual tuning** -- autoloop runs 160k+ experiments/hour

2. **Ensemble over single model** -- WBF-fused YOLO detectors + classifier re-ranking
3. **Bayesian over frequentist** -- hierarchical priors with regime-conditional calibration
4. **Iterate within round window** -- re-submit improved predictions during ~165 min rounds

Product Requirements

Requirements derived from competition rules and scoring criteria.

Astar Island

Objective

Predict 40x40x6 probability distributions for a Norse civilization simulator across 5 seeds per round.

Scoring

- Entropy-weighted KL divergence, scored 0-100
- Only dynamic cells (entropy > 0.01) count
- Probability floor of 0.01 required (avoids infinite KL)
- Target: >90 average score

Constraints

- 50 simulation queries per round (shared across 5 seeds)
- 15x15 viewport per query
- ~165 minute round window
- Rate limit: 5 sim calls/sec, 2 submit calls/sec

Requirements

- [x] Adaptive exploration strategy (entropy-targeted viewports)
- [x] Hierarchical calibration model (fine/coarse/base/global)
- [x] Regime detection (collapse/moderate/boom)
- [x] Parametric simulator with CMA-ES fitting
- [x] Autonomous parameter optimization (autoloop)
- [x] Real-time monitoring dashboard
- [x] Automatic round detection and submission (daemon)
- [ ] Score consistently >92 on boom rounds

NorgesGruppen Data

Objective

Detect and classify grocery products on store shelf images.

Scoring

- `Final = 0.70 * detection_mAP@0.5 + 0.30 * classification_mAP@0.5`
- Detection: category-agnostic box localization
- Classification: correct product class + box

Constraints

- Docker sandbox: Python 3.11, L4 GPU (24 GB), 300s timeout, offline
- Blocked imports: `os`, `sys`, `subprocess`, `pickle`, `yaml`, etc.
- Max zip: 420 MB, max 3 weight files, max 10 Python files
- 3 submissions per day

Requirements

- [x] YOLOv8x detection with TTA
- [x] Multi-model ensemble with WBF fusion
- [x] ConvNeXt-V2 nano re-classification
- [x] Background rejection (class 356)
- [x] torch.load monkeypatch for PyTorch 2.6.0
- [x] Submission under 420 MB and 300s timeout
- [ ] Test set score > 0.95 combined

Tripletex

Objective

Build HTTPS `/solve` endpoint that executes accounting tasks via Tripletex API.

Scoring

- `correctness * tier_multiplier + efficiency_bonus` (up to 6.0 per task)

- 30 task types, 56 variants each
- 7 languages: nb, en, es, pt, nn, de, fr

Requirements

- [ ] HTTPS endpoint deployment
- [ ] LLM agent for prompt interpretation
- [ ] Tripletex API integration (employees, customers, products, invoices, etc.)
- [ ] Multi-language prompt handling

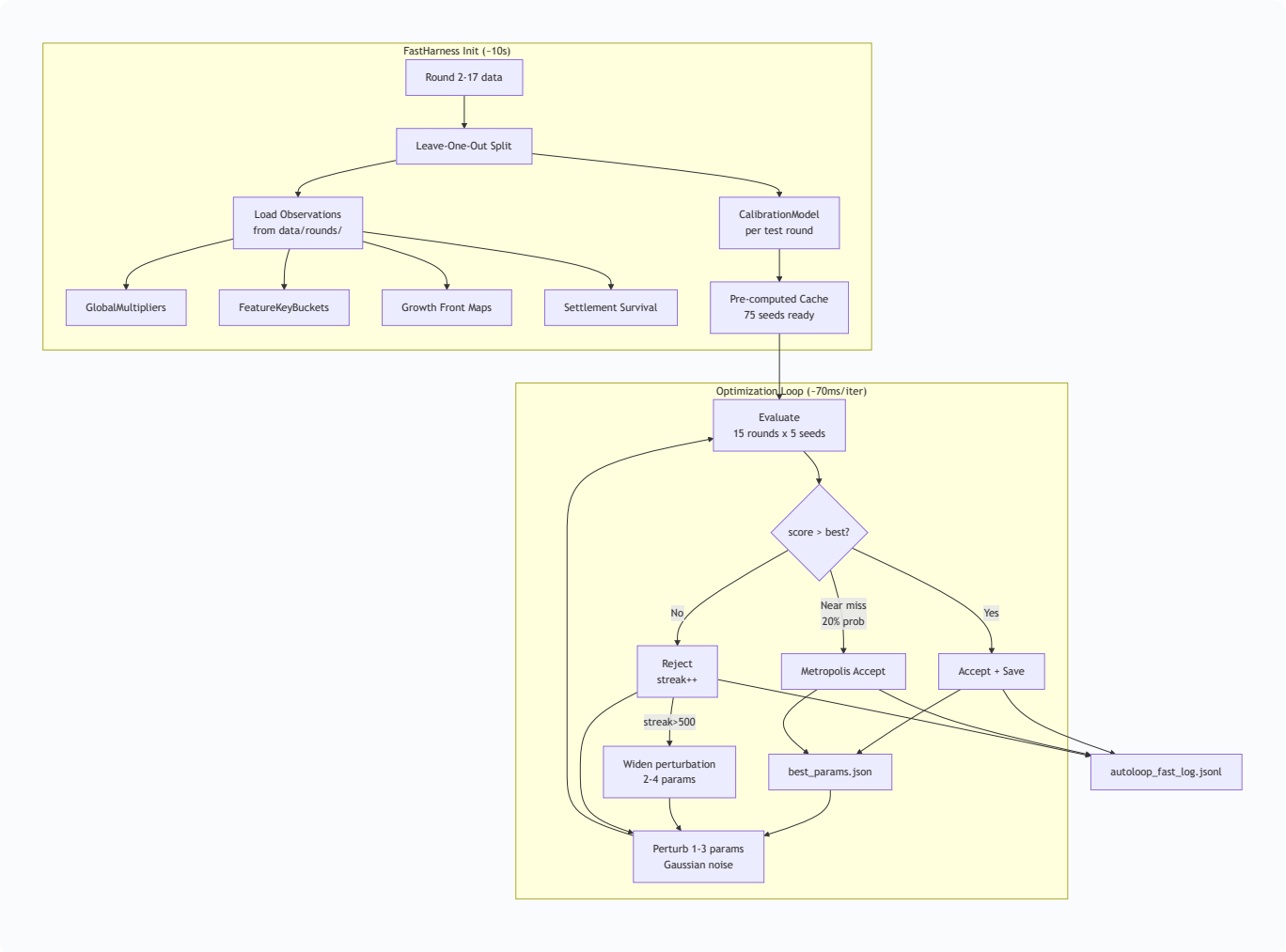
Grocery Bot

Status: Skipped (not scored, warm-up only)

Autoloop -- Deep Technical Reference

Continuous autonomous parameter optimization via Metropolis-Hastings random search with leave-one-out cross-validation backtesting.

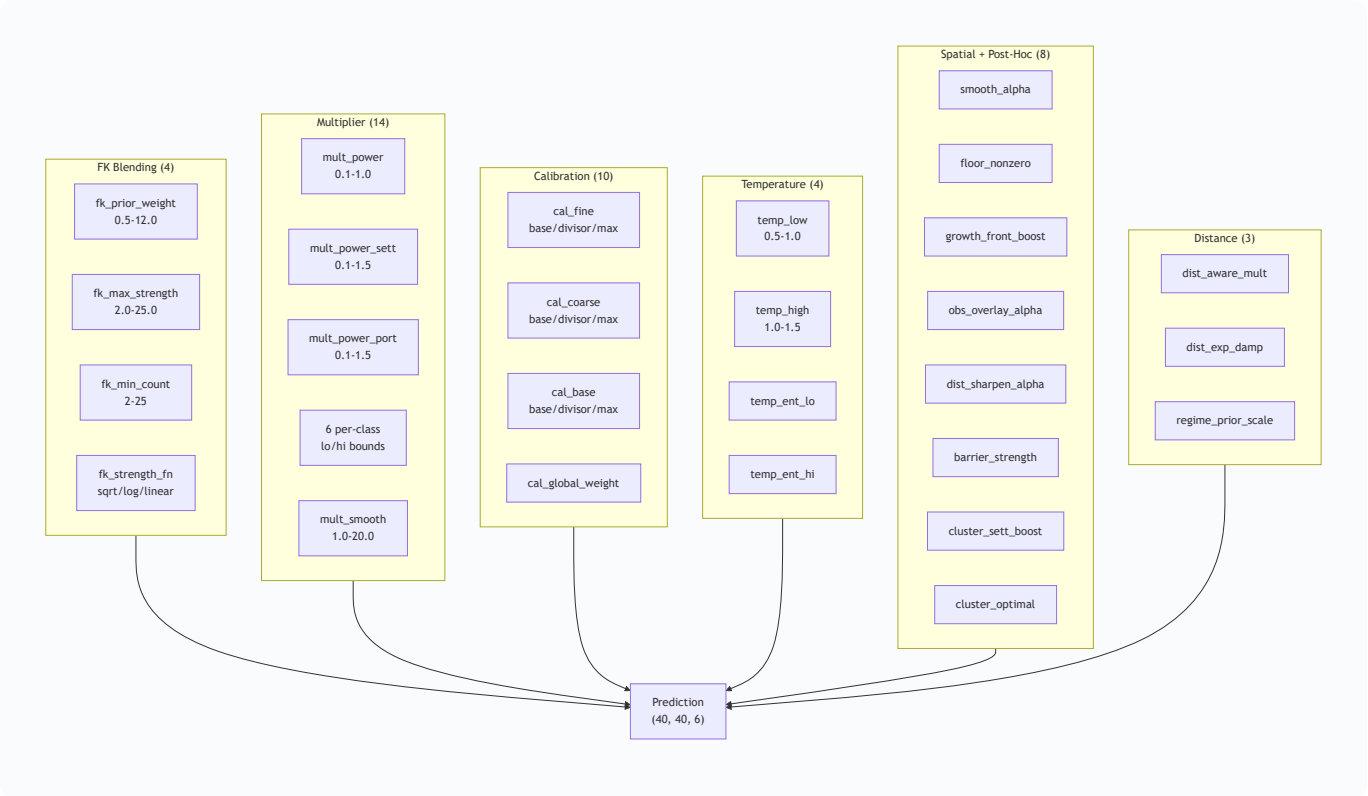
Architecture



Two implementations exist, sharing the same parameter space and experiment log:

	`autoloop.py`	`autoloop_fast.py`
Engine	`autoexperiment.BacktestHarness`	`FastHarness` (inline)
Prediction	`make_pred_fn()` per-cell loops	Vectorized numpy (no Python loops)
Speed	~1s per experiment	~70ms per experiment
Throughput	~3,600/hour	~50,000/hour
Eval rounds	8 rounds	15 rounds (round2-round17, excluding round1/round8)
Seeds	Configurable (1-5)	5 seeds per round

Parameter Space (44 dimensions)



Defined in `autoloop.py:PARAM_SPACE` dict. Each parameter has a type, bounds, and step size for perturbation.

FK Blending (4 params)

Parameter	Type	Range	Step	Default	Purpose
`fk_prior_weight`	float	0.5-12.0	0.25	1.5	Weight of calibration prior in FK blend
`fk_max_strength`	float	2.0-25.0	0.5	20.0	Maximum empirical FK strength cap
`fk_min_count`	int	2-25	1	5	Minimum obs count before FK is used
`fk_strength_fn`	cat	sqrt/log/linear	--	sqrt	Strength scaling function for FK count

Blending formula:

```
strength = min(fk_max_strength, fn(count))  # fn = sqrt, log1p*2, or count*0.1
blended = pred * fk_prior_weight + empirical * strength
blended /= blended.sum()  # renormalize
```

Multiplier System (14 params)

Controls how observed-vs-expected class ratios adjust predictions.

Parameter	Type	Range	Step	Default	Purpose
`mult_power`	float	0.1-1.0	0.02	0.3	Base dampening power for ratio
`mult_power_sett`	float	0.1-1.5	0.02	0.3	Settlement-specific power override
`mult_power_port`	float	0.1-1.5	0.02	0.3	Port-specific power override
`mult_smooth`	float	1.0-20.0	0.5	5.0	Laplace smoothing constant
`mult_sett_lo/hi`	float	0.02-5.0	varies	0.15/2.0	Settlement multiplier clamp bounds
`mult_port_lo/hi`	float	0.02-5.0	varies	0.15/2.0	Port multiplier clamp bounds
`mult_forest_lo/hi`	float	0.2-2.5	varies	0.5/1.8	Forest multiplier clamp bounds
`mult_empty_lo/hi`	float	0.5-1.5	0.02	0.75/1.25	Empty multiplier clamp bounds

Multiplier formula:

```
ratio = (observed + smooth) / (expected + smooth)
mult = ratio^power                    # dampened ratio
mult[class] = clip(mult, lo, hi)      # per-class clamping
pred *= mult                          # apply
```

Distance-Aware Multiplier (2 params)

Parameter	Type	Range	Default	Purpose
`dist_aware_mult`	bool	T/F	True	Enable distance-aware dampening
`dist_exp_damp`	float	0.1-0.9	0.4	How much to dampen multiplier at distance>0

```
# Settlement cells (dist=0): full multiplier
# Expansion cells (dist>=1): dampened multiplier
mult_expansion[c] = 1.0 + (mult[c] - 1.0) * dist_exp_damp
```

Calibration Weights (10 params)

Controls the hierarchical calibration model's weighting at each level.

Parameter	Type	Range	Step	Default	Purpose
`cal_fine_base`	float	0.3-3.0	0.1	1.0	Base weight for fine-level FK
`cal_fine_divisor`	float	30-500	10	120.0	Count divisor for fine weight scaling
`cal_fine_max`	float	1.0-10.0	0.25	4.0	Maximum fine weight cap
`cal_coarse_base`	float	0.2-2.0	0.1	0.75	Base weight for coarse-level FK
`cal_coarse_divisor`	float	50-500	10	200.0	Count divisor for coarse weight scaling
`cal_coarse_max`	float	1.0-8.0	0.25	3.0	Maximum coarse weight cap
`cal_base_base`	float	0.1-2.0	0.05	0.5	Base weight for terrain-only level
`cal_base_divisor`	float	200-3000	50	1000.0	Count divisor for base weight scaling
`cal_base_max`	float	0.5-5.0	0.1	1.5	Maximum base weight cap
`cal_global_weight`	float	0.05-2.0	0.05	0.4	Fixed weight for global fallback

Weight formula for each level:

```
weight = min(max_weight, base_weight + observation_count / divisor)
```

Temperature Scaling (4 params)

Entropy-aware temperature applied to sharpen or soften predictions.

Parameter	Type	Range	Default	Purpose
`temp_low`	float	0.5-1.0	1.0	Temperature for low-entropy (certain) cells
`temp_high`	float	1.0-1.5	1.15	Temperature for high-entropy (uncertain) cells
`temp_ent_lo`	float	0.1-0.5	0.2	Entropy threshold for "low" classification
`temp_ent_hi`	float	0.6-1.5	1.0	Entropy threshold for "high" classification

```
T = T_low + clip((cell_entropy - ent_lo) / (ent_hi - ent_lo)) * (T_high - T_low)
pred = pred^(1/T)  # T<1 sharpens, T>1 softens
```

Post-Hoc Corrections (8 params)

Parameter	Type	Range	Default	Purpose
`smooth_alpha`	float	0.0-0.5	0.15	Spatial smoothing blend (settlement/ruin only)
`floor_nonzero`	float	0.001-0.015	0.008	Probability floor for nonzero classes
`growth_front_boost`	float	0.0-1.0	0.0	Boost factor for young settlement expansion
`obs_overlay_alpha`	float	0.0-100.0	0.0	Dirichlet pseudo-count for obs overlay
`dist_sharpen_alpha`	float	0.0-1.0	0.0	Per-distance rate correction strength
`barrier_strength`	float	0.0-1.0	0.0	Terrain barrier penalty strength
`cluster_sett_boost`	float	0.0-1.0	0.0	Cluster density cooperative boost
`cluster_optimal`	float	0.5-5.0	2.0	Optimal cluster density (inverted-U peak)

FastHarness Pre-computation

`FastHarness.__init__()` runs once on startup (~5-10s) and caches everything that doesn't change with parameters:

Per-Round (15 rounds):

1. Load `round_detail.json` and observations from `data/rounds/{uuid}/`
2. Build `GlobalMultipliers` and `FeatureKeyBuckets` from observations
3. Estimate vigor (settlement % on dynamic cells) for regime-conditional calibration
4. Build `CalibrationModel` from all rounds EXCEPT the test round (LOO)
5. If regime-conditional: weight each training round by Gaussian similarity to test vigor

Per-Seed (5 per round, 75 total):

1. Load `analysis_seed_{i}.json` (ground truth)
2. Pre-compute `feature_keys` (6-tuple per cell)
3. Pre-compute `idx_grid` (integer index mapping FK -> unique key index)
4. Pre-compute `coastal` mask, `static_mask`, `dynamic_mask`
5. Pre-compute `cluster_density` (settlements within Manhattan r=5)
6. Pre-compute `openness` (diffusion-based terrain accessibility)
7. Pre-compute `growth_front` map from observation settlement populations
8. Pre-compute `obs_overlay` (Dirichlet observation counts per cell)
9. Pre-compute `sett_survival` (alive/dead counts per initial settlement)
10. Pre-compute `dist_map` (Manhattan distance to nearest initial settlement)

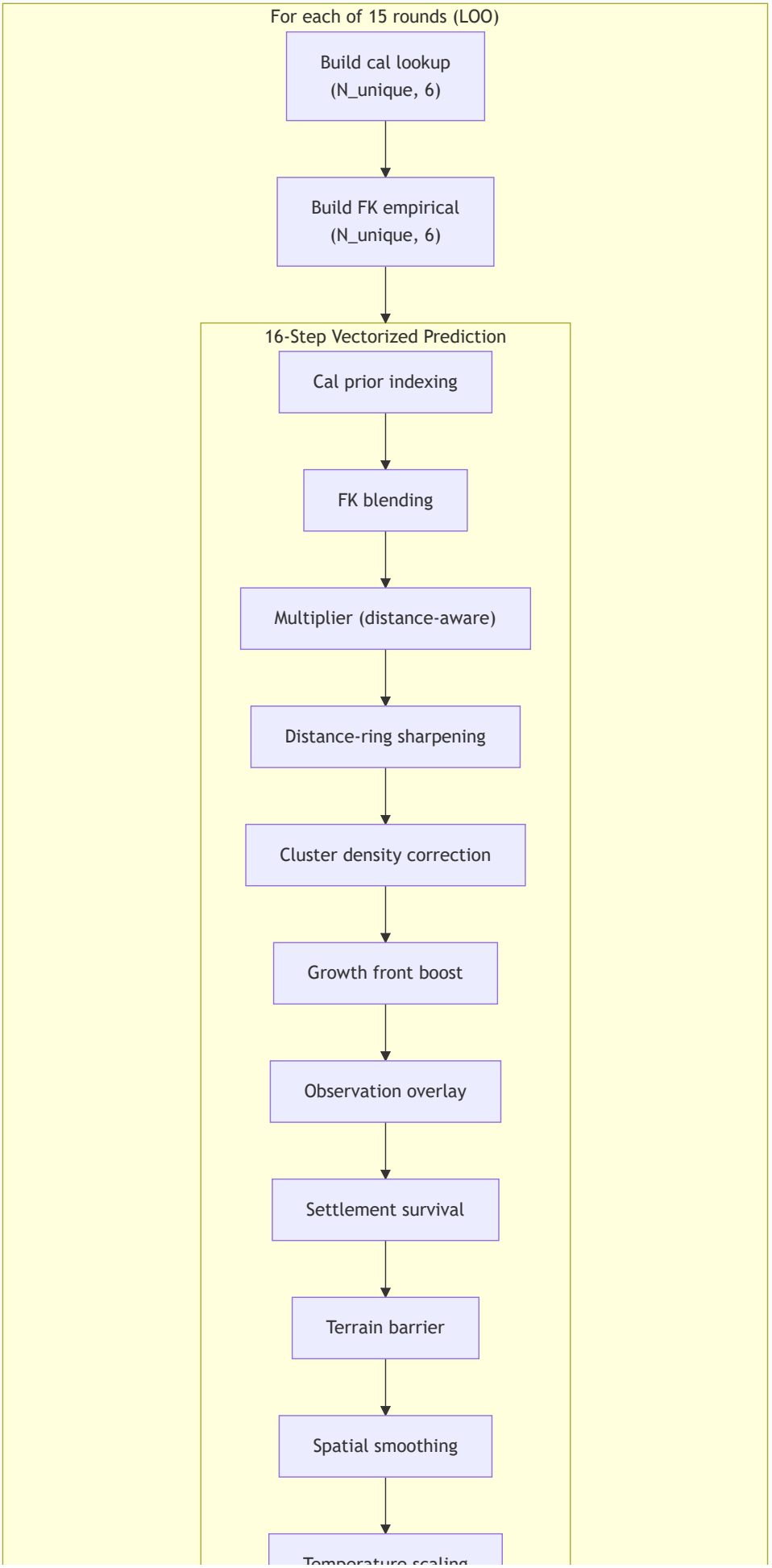
Per-Round Shared:

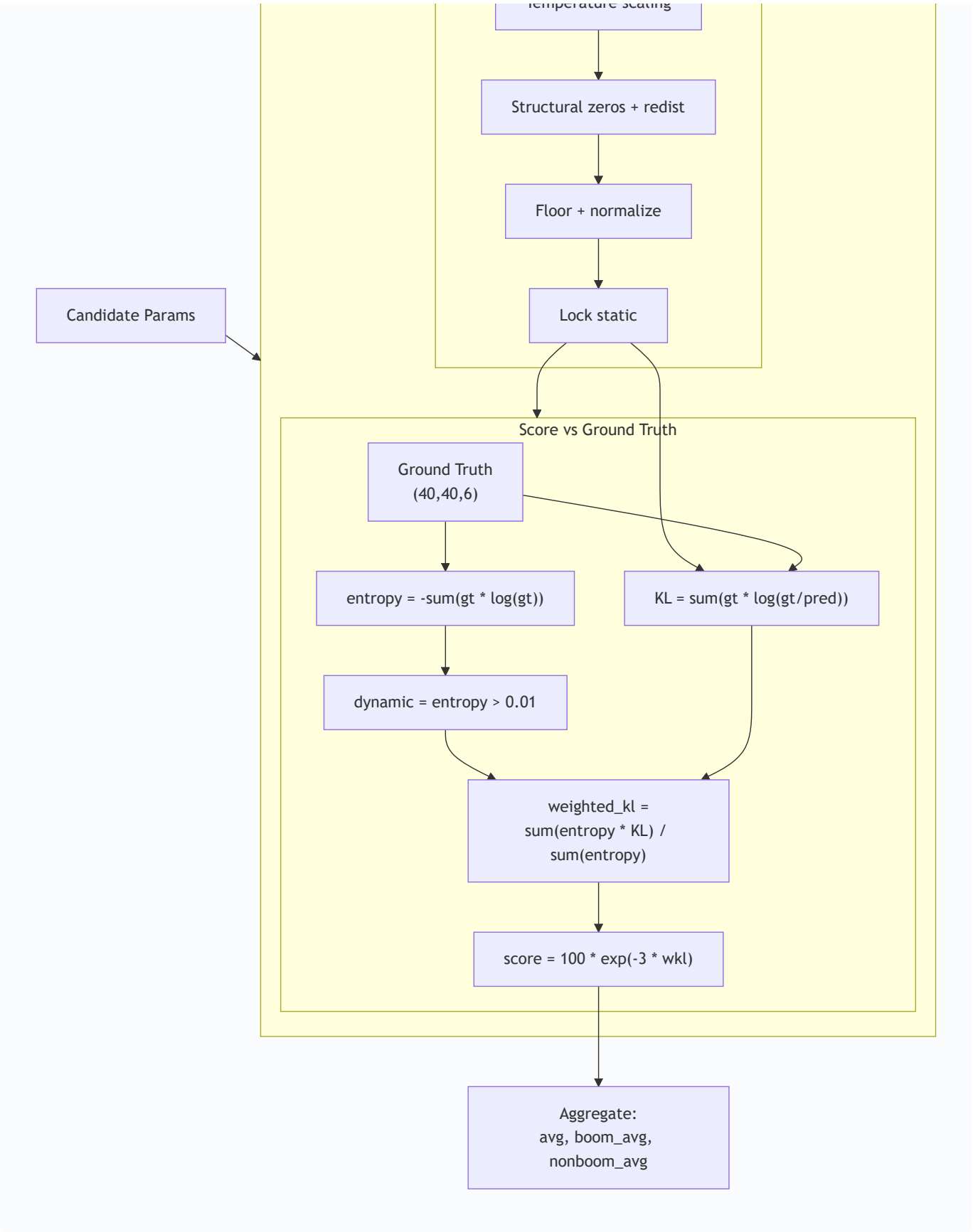
- `dist_obs_rates`: pooled per-distance observation rates across all seeds
- `est_vigor`: estimated vigor for regime-conditional calibration
- `CalibrationModel`: regime-weighted or uniform-weighted from LOO training rounds

---

Evaluation Pipeline (`FastHarness.evaluate()`)

Called for every candidate parameter set. Returns dict with per-round and aggregate scores.


For each of 15 rounds (LOO)
Build cal lookup (N_unique, 6)
Build FK empirical (N_unique, 6)
16-Step Vectorized Prediction
Cal prior indexing
FK blending
Multiplier (distance-aware)
Distance-ring sharpening
Cluster density correction
Growth front boost
Observation overlay
Settlement survival
Terrain barrier
Spatial smoothing


Structural zeros + redist
Floor + normalize
Lock static
Candidate Params
Score vs Ground Truth
Ground Truth
(40,40,6)
entropy = -sum(gt * log(gt))
KL = sum(gt * log(gt/pred))
dynamic = entropy > 0.01
weighted_kl =
sum(entropy * KL) /
sum(entropy)
score = 100 * exp(-3 * wkl)
Aggregate:
avg, boom_avg,
nonboom_avg

Steps per seed (inlined, no function calls):

```
1. Build calibration lookup table: (N_unique_keys, 6)
2. Build FK empirical lookup: (N_unique_keys, 6) + (N_unique_keys,) counts
3. Index into lookup tables via idx_grid -> (40, 40, 6) prediction
4. FK bucket blending (vectorized)
5. Global multiplier (distance-aware or uniform)
6. Distance-ring sharpening (per-distance rate correction)
7. Cluster density multiplier (inverted-U spatial correction)
8. Growth front boost (young settlement expansion)
9. Observation overlay (Dirichlet-Multinomial conjugate update)
10. Settlement survival constraints (per-initial-settlement)
11. Terrain barrier correction (diffusion-based openness)
12. Selective spatial smoothing (3x3 uniform filter on sett/ruin)
13. Entropy-weighted temperature scaling
14. Structural zeros + proportional redistribution
15. Floor + renormalize
16. Lock static cells (ocean/mountain)
17. Compute score vs ground truth
```

Score Computation

```
def compute_score(gt, pred):
    entropy = -sum(gt * log(gt))           # per-cell entropy
    dynamic = entropy > 0.01               # static cells excluded
    kl = sum(gt * log(gt / pred))          # per-cell KL divergence
    weighted_kl = sum(entropy[dynamic] * kl[dynamic]) / sum(entropy[dynamic])
    return 100 * exp(-3 * weighted_kl)     # 0-100 scale
```

Aggregate Scoring

```
results["avg"] = mean(all_round_scores)
results["boom_avg"] = mean(scores for boom rounds)      # round6,7,11,14,17
results["nonboom_avg"] = mean(scores for non-boom rounds)
```

Optimization Loop (`main()`)

```
harness = FastHarness(seeds_per_round=5, regime_conditional=True)
log = ExperimentLog(LOG_PATH)
best_score = log.best_score
best_params = log.best_params
reject_streak = 0

while True:
    # 1. Propose candidate
    n_changes = 1-3 (widened to 2-4 after 500 rejections)
    name, candidate = perturb_params(best_params, n_changes)

    # 2. Evaluate (LOO backtest)
    results = harness.evaluate(candidate)
    avg = results["avg"]

    # 3. Accept/reject
    if avg > best_score:
        # ACCEPT: strict improvement
        best_score = avg
        best_params = candidate
        reject_streak = 0
        save_best_params_json(candidate, avg)
    elif random() < 0.2 and avg > best_score - 0.05:
        # METROPOLIS ACCEPT: near-miss exploration (20% chance)
        best_params = candidate  # but don't update best_score
        reject_streak = 0
    else:
        # REJECT
        reject_streak += 1

    # 4. Log to JSONL
    log.append({id, name, params, scores, accepted, timestamp})

    # 5. Adaptive perturbation width
    if reject_streak > 500:
        n_changes = random.choices([2,3,4], weights=[0.4,0.4,0.2])
        # Wider exploration to escape local optimum
```

Perturbation Strategy (`perturb_params()`)

```
def perturb_params(base, n_changes=None):
    if n_changes is None:
        n_changes = random.choices([1, 2, 3], weights=[0.5, 0.35, 0.15])

    keys = random.sample(PARAM_SPACE.keys(), n_changes)

    for key in keys:
        spec = PARAM_SPACE[key]
        if spec.type == "float":
            delta = gauss(0, spec.step * 2)       # Normal perturbation
            new = clip(old + delta, spec.lo, spec.hi)
        elif spec.type == "int":
            delta = randint(-2, 2)
            new = clip(old + delta, spec.lo, spec.hi)
        elif spec.type == "cat":
            if random() < 0.3:                     # 30% chance to flip
                new = random.choice(spec.choices)
```

Experiment Log Format

JSONL file at `data/autoloop_fast_log.jsonl`, one entry per experiment:

```
{
    "id": 770671,
    "name": "fk_prior_weight=5.8555, T_high=1.0011",
    "params": {"fk_prior_weight": 5.8555, ...},
    "scores_full": {
        "avg": 89.389,
        "boom_avg": 87.714,
        "nonboom_avg": 90.226,
        "round2": 90.1, "round3": 89.7, ...
    },
    "accepted": true,
    "timestamp": "2026-03-21T22:49:00Z"
}
```

`best_params.json` Contract

Written atomically on every improvement. Read by `predict_gemini.py` at prediction time.

```
{
    "source": "autoloop_fast",
    "score_avg": 89.389,
    "score_boom": 87.714,
    "score_nonboom": 90.226,
    "experiment_id": 770671,
    "timestamp": "2026-03-21T22:49:00Z",
    "fk_prior_weight": 5.8555,
    "fk_max_strength": 17.5003,
    "T_high": 1.0011,
    "smooth_alpha": 0.4808,
    ...
}
```

Regime-Conditional Calibration

When `regime_conditional=True`, the `CalibrationModel` weights training rounds by vigor similarity to the test round:

```
for train_round in train_rounds:
    vigor_similarity = exp(-(train_vigor - test_vigor)^2 / (2 * sigma^2))
    vigor_similarity = max(vigor_similarity, 0.05)  # floor to prevent exclusion
    cal.add_round(train_round_dir, weight=vigor_similarity)
```

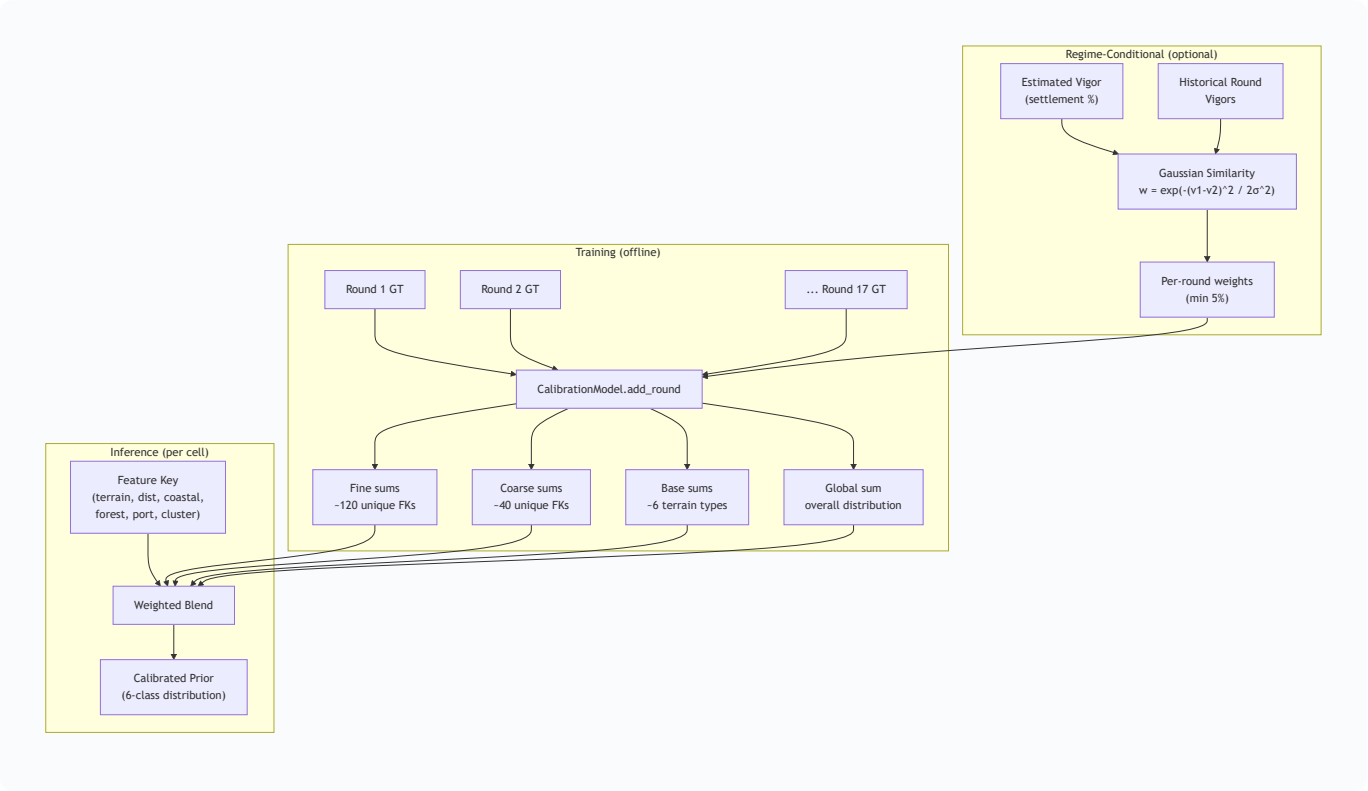
sigma = 0.06 (default). This means:

- A boom training round (vigor=0.20) gets full weight when testing a boom round
- A collapse training round (vigor=0.01) gets near-zero weight when testing a boom
- Every round gets at least 5% weight (floor)

Calibration Model -- Technical Reference

Hierarchical feature-key-based prior model trained on 17 rounds of historical ground truth. The backbone of the prediction system.

Data Flow



Hierarchy

```
┌──────────────────────────────────────────┐
│ Global:  overall class distribution      │
│ Weight: cal_global_weight (fixed 0.4)    │
└──────────────────────────────────────────┘
                ▲
┌──────────────────────────────────────────┐
│ Base:    (terrain_code,)                 │
│ Weight: cal_base_base + count/divisor    │
│ Keys: ~6 unique terrain types            │
└──────────────────────────────────────────┘
                ▲
┌──────────────────────────────────────────┐
│ Coarse:  (terrain, dist, coastal, port)  │
│ Weight: cal_coarse_base + count/divisor  │
│ Keys: ~40 unique combinations            │
└──────────────────────────────────────────┘
                ▲
┌──────────────────────────────────────────┐
│ Fine:    (terrain, dist, coastal,        │
│           forest_neighbors, port, cluster)│
│ Weight: cal_fine_base + count/divisor     │
│ Keys: ~120 unique combinations           │
└──────────────────────────────────────────┘
```

Feature Key Definition

```
FeatureKey = (
    terrain_code: int,      # Raw terrain code (0, 1, 2, 3, 4, 5, 10, 11)
    dist_bucket: int,       # Manhattan distance to nearest settlement, bucketed
    coastal: bool,          # Adjacent to ocean cell (4-connected)
    forest_neighbors: int,  # Count of forest neighbors (0-3, capped)
    has_port_flag: int,     # -1=not settlement, 0=settlement no port, 1=port
    cluster_bucket: int,    # 0=isolated, 1=sparse(1-2), 2=dense(3+)
)
```

Distance Bucketing

Distance	Bucket	Rationale
0	0	On a settlement
1	1	Immediate neighbor
2	2	Close expansion zone
3	3	Moderate expansion
4-5	4	Nearby expansion zone
6-8	5	Moderate distance
9+	6	Far from settlements

The d=4-5 vs d=6-8 split was added because cells at d=4-5 behave very differently from d=8+ but were previously lumped together, causing the biggest prediction errors.

CalibrationModel Class

Data Structures

```
class CalibrationModel:
    fine_sums: dict[FeatureKey, ndarray(6)]     # Accumulated GT probability sums
    fine_counts: dict[FeatureKey, float]         # Observation count per FK
    coarse_sums: dict[tuple(4), ndarray(6)]     # Coarse level (drop forest, cluster)
    coarse_counts: dict[tuple(4), float]
    base_sums: dict[int, ndarray(6)]            # Terrain-only level
    base_counts: dict[int, float]
    global_sum: ndarray(6)                      # Overall class sum
    global_count: float
    global_probs: ndarray(6)                    # Normalized global prior
```

Training (`add_round()`)

For each round directory containing `round_detail.json` and `analysis_seed_*.json`:

```
for seed in range(seeds_count):
    terrain = analysis["initial_grid"]      # (40, 40) int
    ground_truth = analysis["ground_truth"]  # (40, 40, 6) float
    settlements = detail["initial_states"][seed]["settlements"]
    feature_keys = build_feature_keys(terrain, settlements)

    for y, x in all_cells:
        fk = feature_keys[y][x]
        gt = ground_truth[y, x] * weight  # weight for regime-conditional

        fine_sums[fk] += gt
        fine_counts[fk] += weight
        coarse_sums[(fk[0], fk[1], fk[2], fk[4])] += gt
        coarse_counts[coarse_key] += weight
        base_sums[fk[0]] += gt
        base_counts[fk[0]] += weight
        global_sum += gt
        global_count += weight
```

Data volume: 17 rounds x 5 seeds x 40x40 = 136,000 cells total.

Inference (`prior_for()`)

```
def prior_for(fk):
    vector = zeros(6)
    total_weight = 0

    if fine_count[fk] > 0:
        fine_w = min(4.0, 1.0 + fine_count / 120.0)
        vector += fine_w * normalize(fine_sums[fk])
        total_weight += fine_w

    if coarse_count[coarse_key] > 0:
        coarse_w = min(3.0, 0.75 + coarse_count / 200.0)
        vector += coarse_w * normalize(coarse_sums[coarse_key])
        total_weight += coarse_w

    if base_count[terrain] > 0:
        base_w = min(1.5, 0.5 + base_count / 1000.0)
        vector += base_w * normalize(base_sums[terrain])
        total_weight += base_w

    vector += 0.4 * global_probs  # regularizer
    total_weight += 0.4

    return floor_and_renormalize(vector / total_weight)
```

Note: The autoloop overrides these hardcoded weights with tunable params (`cal_fine_base` , `cal_fine_divisor` , etc.)

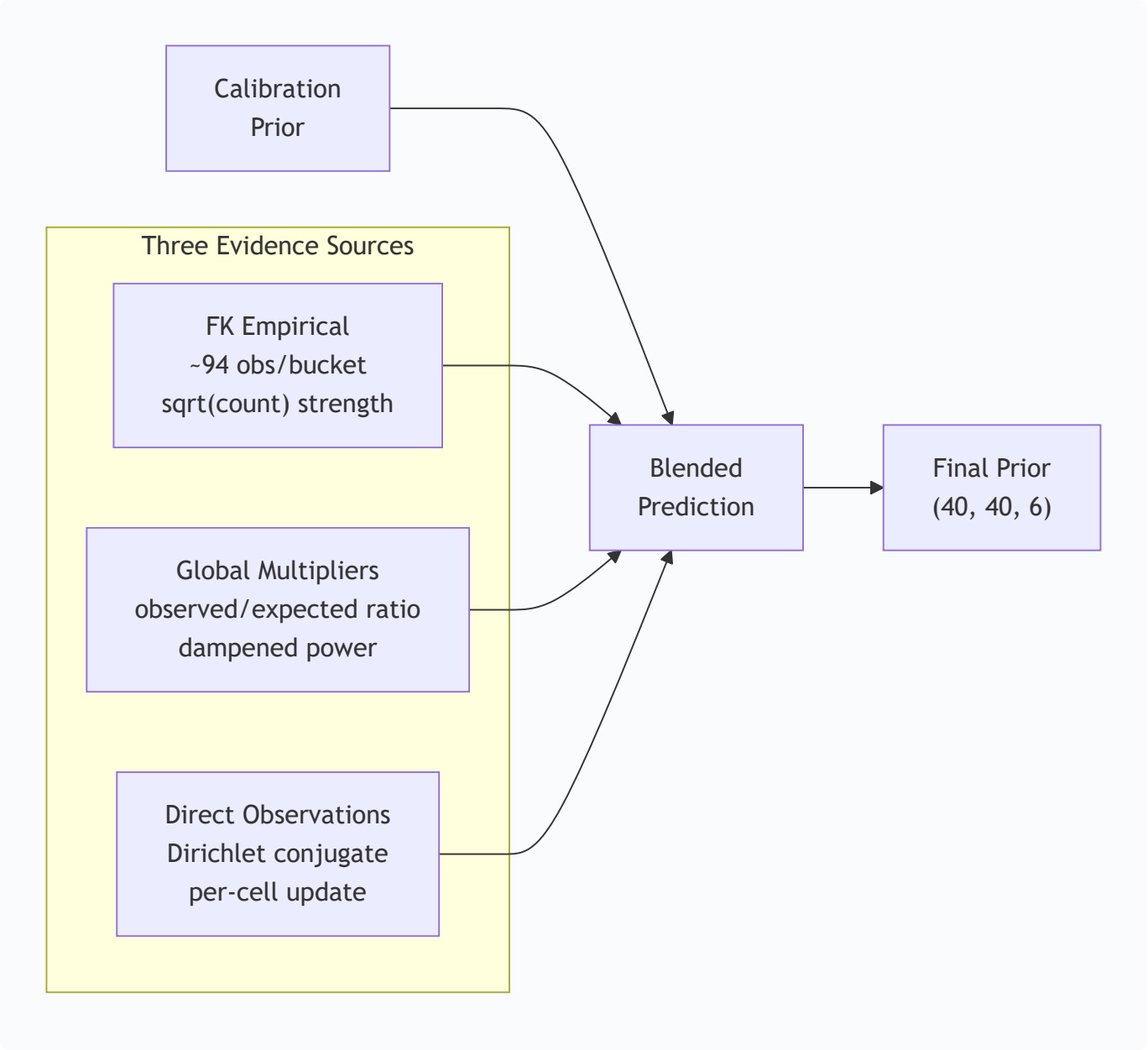
Regime-Conditional Calibration

When estimated vigor is available, training rounds are weighted by similarity:

```
@staticmethod
def compute_round_vigor(round_dir):
    # Mean settlement probability on dynamic cells across all seeds
    for seed in range(seeds_count):
        dynamic = (terrain != ocean) & (terrain != mountain)
        sett_probs.append(gt[dynamic, settlement_class].mean())
    return mean(sett_probs)

# During CalibrationModel construction:
for train_round in train_rounds:
    similarity = exp(-(train_vigor - test_vigor)^2 / (2 * sigma^2))
    similarity = max(similarity, 0.05)  # minimum floor
    cal.add_round(train_round, weight=similarity)
```

sigma=0.06 means rounds with similar settlement densities contribute most. A collapse training round (vigor=0.01) gets ~5% weight when predicting a boom round (vigor=0.20).

Observation-Based Enrichment



The calibration prior is enriched with live observation data through three mechanisms:

1. FeatureKeyBuckets (FK empirical)

Pool observations by feature key across all seeds.

- ~11,250 total observations (50 queries x 15x15 viewport x 5 seeds)
- ~120 unique FKs -> ~94 observations per bucket
- Empirical distribution with sqrt-count strength scaling

2. GlobalMultipliers

Track class ratio: observed/expected.

- Detects regime shift: collapse (ratio << 1), boom (ratio >> 1)
- Applied as dampened multiplicative correction

3. Direct Observation Overlay

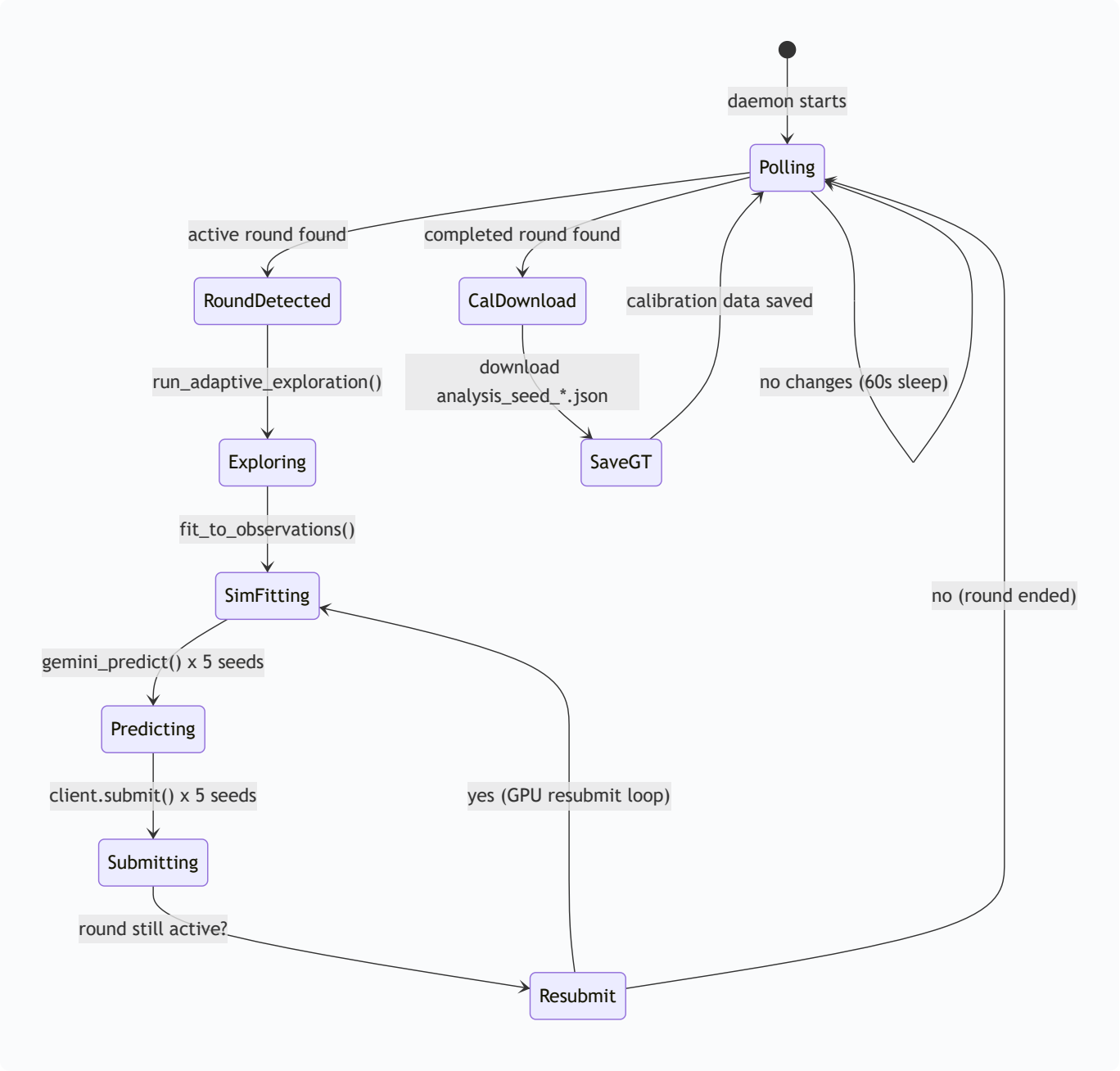
Dirichlet-Multinomial conjugate update on directly observed cells.

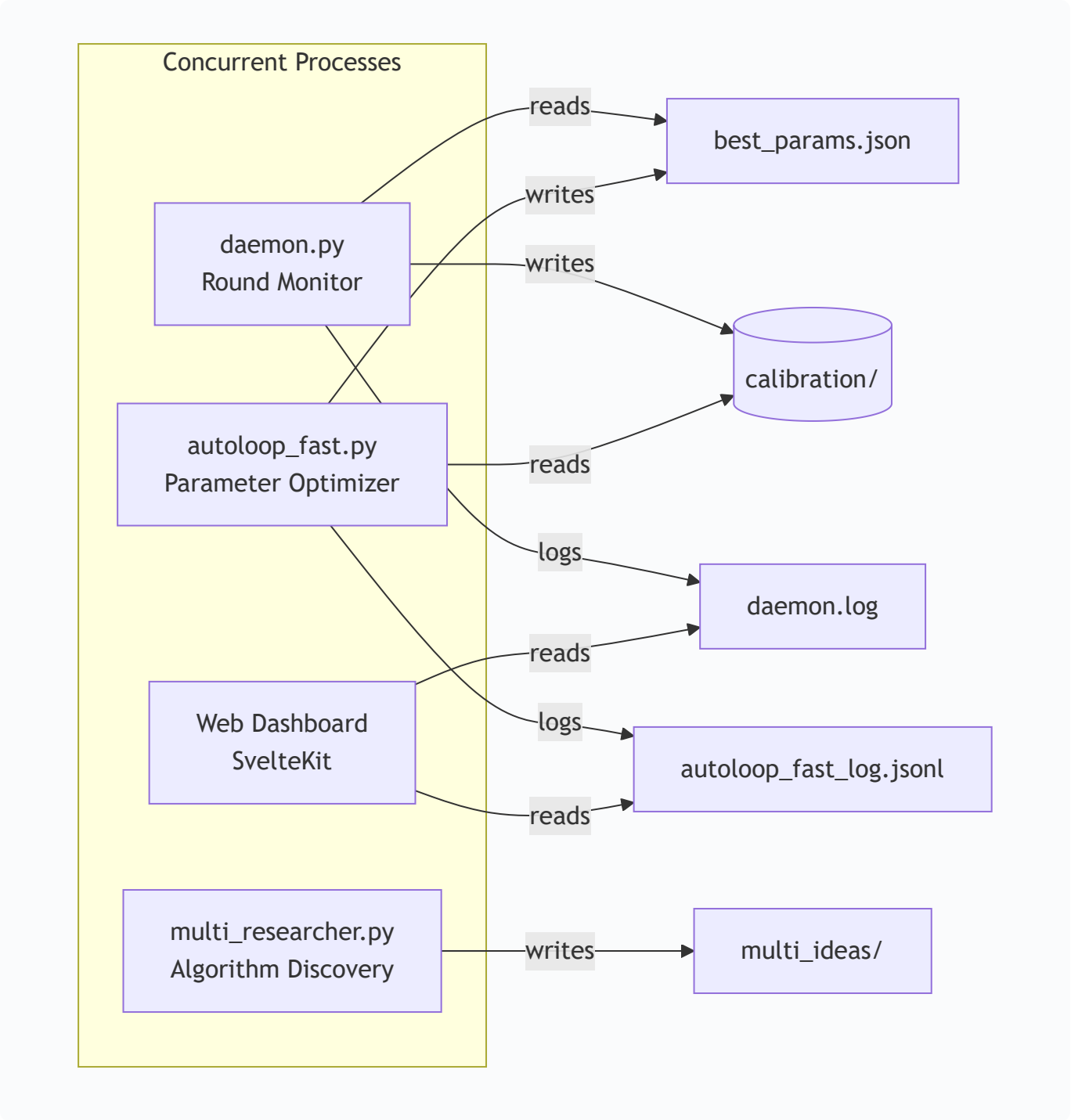
- Per-cell, no spatial spreading
- Weight controlled by `obs_pseudo` parameter

Daemon System -- Technical Reference

Autonomous round detection, exploration, prediction, and iterative submission. Runs unattended 24/7.

Architecture

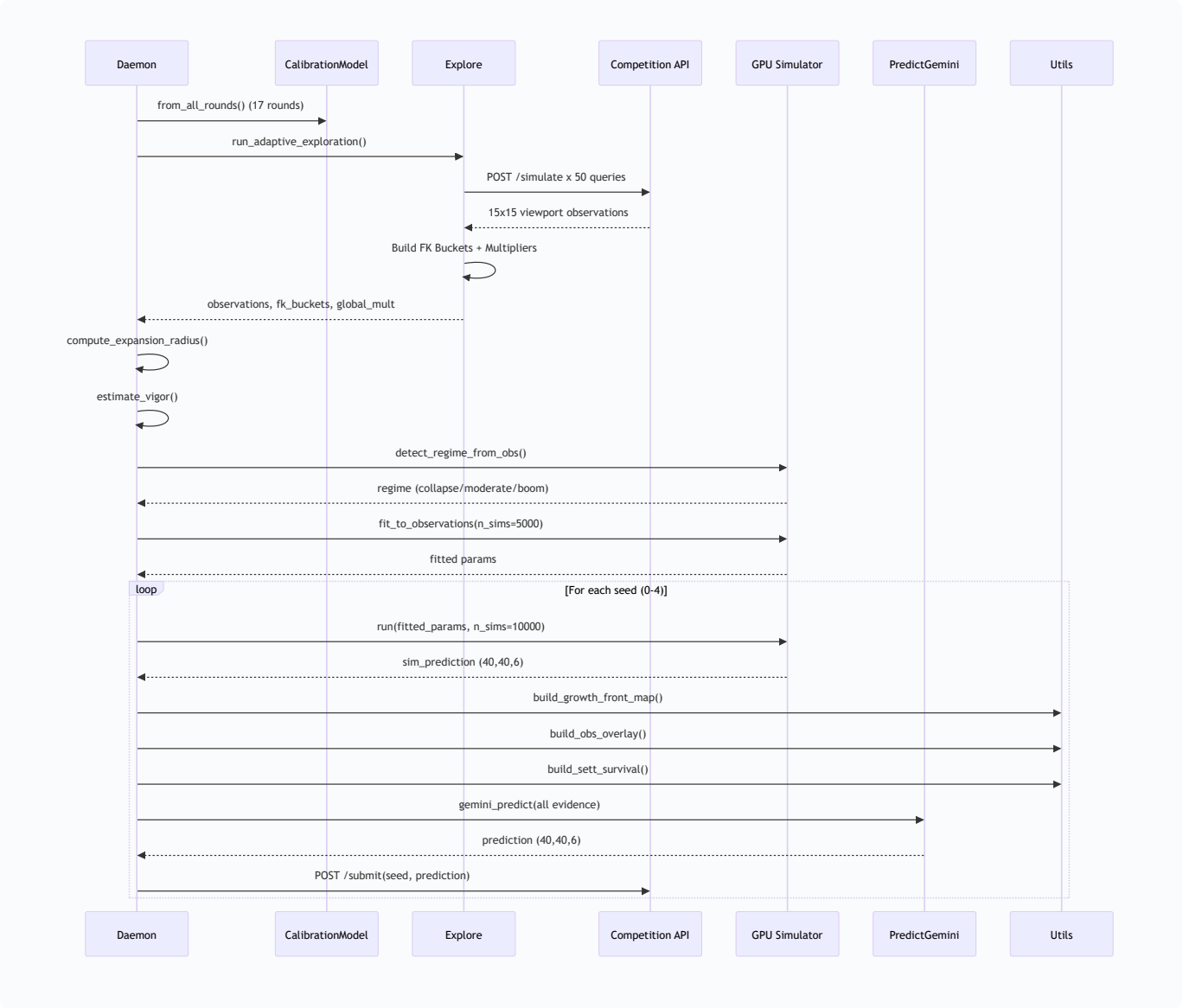



Concurrent Processes
daemon.py
Round Monitor
autoloop_fast.py
Parameter Optimizer
Web Dashboard
SvelteKit
multi_researcher.py
Algorithm Discovery
reads
writes
writes
reads
logs
reads
logs
reads
writes
best_params.json
calibration/
daemon.log
autoloop_fast_log.jsonl
multi_ideas/

Round Monitor

```
while True:
    rounds = client.get_rounds()
    for round in rounds:
        if round.status == "active" and round.id not in submitted:
            run_submission(client, round.id, detail)
            submitted.add(round.id)
        elif round.status == "completed" and round.number not in calibrated:
            download_round_analysis(client, round.id, round.number)
            calibrated.add(round.number)
    sleep(check_interval)  # default 60s
```

Submission Pipeline (`run_submission()`)



Step 1: Load Fresh Calibration

```
cal = CalibrationModel.from_all_rounds()
predict._calibration = cal  # monkey-patch global singleton
```

Step 2: Adaptive Exploration

```
exploration = run_adaptive_exploration(client, round_id, detail)
# Returns: global_multipliers, fk_buckets, multi_store, variance_regime, observations
```

Step 3: Compute Expansion Radius

```
exp_radius = compute_expansion_radius(obs_list, detail)
# Returns: dict {distance: (sett_count, total_count)} per seed
```

Iterates through observations, computes Manhattan distance from each observed cell to nearest initial settlement, then accumulates settlement counts per distance bucket.

Step 4: Estimate Vigor

```
est_vigor = sett_observations / total_dynamic_observations
# Used for regime-conditional calibration
```

Step 5: Simulator Inference

```
# Detect regime from observations
regime = detect_regime_from_obs(obs_list, terrain)

# Choose GPU or CPU
use_gpu = torch.cuda.is_available()
fit_sims = 5000 if use_gpu else 500    # MC samples per CMA-ES eval
pred_sims = 10000 if use_gpu else 2000  # MC samples for final prediction

# Fit simulator parameters
sim_params, _ = fit_to_observations(rd, obs_list, n_sims=fit_sims,
                                    max_evals=200, use_gpu=use_gpu)

# Generate predictions per seed
for seed_idx in range(5):
    sim_predictions[seed_idx] = sim.run(sim_params, n_sims=pred_sims)
```

Step 6: Build Per-Seed Evidence

```
for seed_idx in range(5):
    growth_front_maps[seed_idx] = build_growth_front_map(seed_obs, terrain)
    obs_overlays[seed_idx] = build_obs_overlay(obs_list, terrain, seed_idx)
    sett_survivals[seed_idx] = build_sett_survival(obs_list, settlements, seed_idx)
```

Step 7: Predict and Submit

```
for seed_idx in range(5):
    prediction = gemini_predict(
        state, global_mult, fk_buckets,
        sim_pred=sim_predictions.get(seed_idx),
        sim_alpha=adaptive_alpha,
        growth_front_map=growth_front_maps.get(seed_idx),
        obs_overlay=obs_overlays.get(seed_idx),
        sett_survival=sett_survivals.get(seed_idx),
        est_vigor=est_vigor,
        obs_expansion_radius=exp_radius,
    )
    errors = validate_prediction(prediction)
    if errors:
        prediction = apply_floor(prediction)
    client.submit(round_id, seed_idx, prediction.tolist())
```

GPU Resubmission Loop (`gpu_resubmit_round()`)

Called periodically while round is active. Each iteration uses a different strategy.

Strategies by Iteration

Iteration	Strategy
0	Base: CMA-ES with default budget + alpha=0.25
1	Tighter sigma (0.2), more evaluations
2	Different warm start (KNN neighbors only)
3+	Wider sigma (0.8), exploration mode

Iterative Improvement Process

```
while round_is_active():
    # 1. Reload observations
    observations = load_all_obs(round_id)

    # 2. Rebuild multipliers and FK buckets (may have changed)
    gm = GlobalMultipliers()
    fk = FeatureKeyBuckets()
    for obs in observations:
        accumulate(gm, fk, obs)

    # 3. Re-fit simulator with different strategy per iteration
    sim_params = fit_varied_strategy(iteration)

    # 4. Predict and submit
    for seed in range(5):
        pred = gemini_predict(state, gm, fk,
                              sim_pred=sim.run(sim_params),
                              sim_alpha=adaptive_alpha)
        client.submit(round_id, seed, pred)

    iteration += 1
    wait(submission_cooldown)
```

Calibration Download (`download_round_analysis()`)

```
def download_round_analysis(client, round_id, round_number):
    cal_dir = data/calibration/round{N}/

    if analysis_seed_0.json exists:
        return  # Already downloaded

    detail = client.get_round_detail(round_id)
    save(cal_dir / "round_detail.json", detail)

    for seed in range(seeds_count):
        analysis = client.get_analysis(round_id, seed)
        save(cal_dir / f"analysis_seed_{seed}.json", analysis)
        # Contains: ground_truth (40,40,6), initial_grid, score
```

Logging

All actions logged to `data/daemon.log` with timestamps:

```
[22:15:30] [INFO] Checking for active rounds...
[22:15:31] [INFO]   Round 18 detected (active)
[22:15:31] [INFO]   Calibration: 17 rounds, 136000 cells
[22:15:32] [INFO]   Starting adaptive exploration...
[22:15:45] [INFO]   Variance regime: MODERATE
[22:15:45] [INFO]   Multipliers: sett=0.823, port=0.156, forest=1.102
[22:15:45] [INFO]   Observed expansion radius: {0: (12, 15), 1: (8, 42), ...}
[22:15:46] [INFO]   Using params: prior_w=5.86, T_high=1.00, score_avg=89.389
[22:15:46] [INFO]   Estimated vigor: 0.0712
[22:15:47] [INFO]   Simulator: using GPU
[22:15:48] [INFO]   Simulator regime=moderate, alpha=0.35
[22:16:15] [INFO]   Simulator params fitted: base_surv=-0.47, exp_str=0.52
[22:16:35] [INFO]   Simulator predictions: 5 seeds, 10000 sims each
[22:16:36] [INFO]   Seed 0: ok (sett=0.0682)
[22:16:36] [INFO]   Seed 1: ok (sett=0.0715)
...
[22:16:38] [INFO]   All 5 seeds submitted for R18
```

Error Handling

Error	Handling
API 429 (rate limit)	Exponential backoff with retry
API timeout	Log warning, skip submission, retry next cycle
GPU OOM	Fall back to CPU simulator
Validation errors	Apply floor and re-normalize, then submit
Missing observations	Submit with calibration-only prediction (no obs enrichment)
Simulator fit failure	Log warning, submit without sim blend (sim_alpha=0)

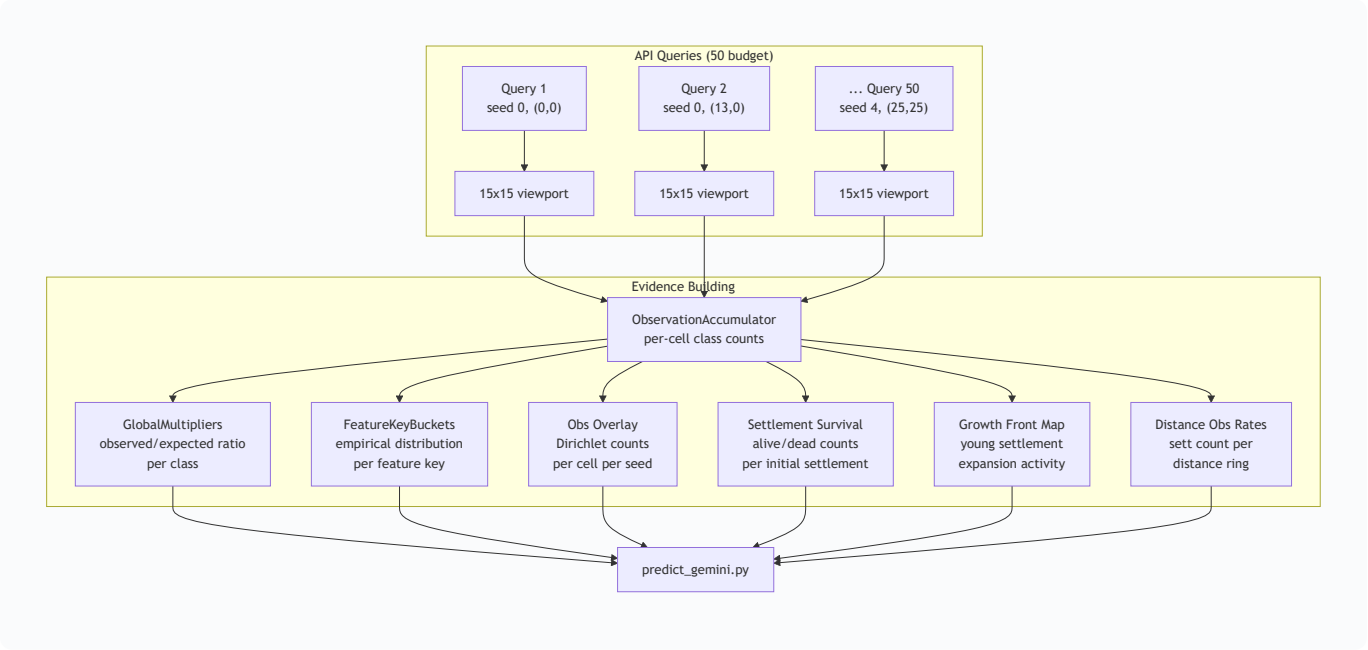
Exploration System -- Technical Reference

Strategic data collection within the 50-query budget per round. Transforms raw API viewport observations into the statistical building blocks used by the prediction engine.

Budget Constraint

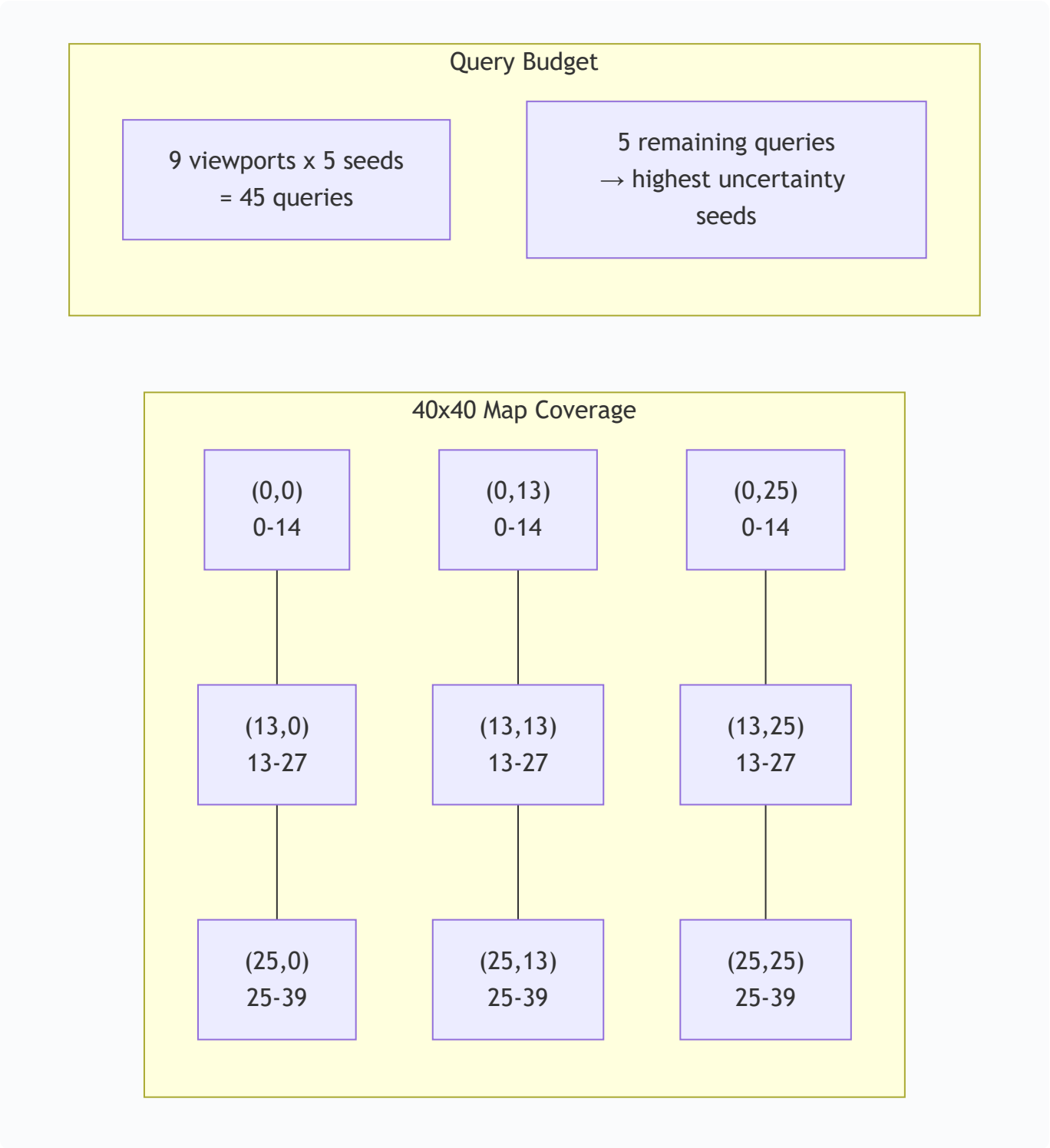
- 50 simulation queries per round, shared across 5 seeds
- Each query returns a 15x15 viewport of the 40x40 map
- Rate limit: 5 queries/second

Data Flow: Observations to Evidence



Exploration Strategies

Grid Strategy (Default)



Fixed 3x3 grid providing ~97% map coverage:

```
GRID_POSITIONS = [
    (0, 0),  (13, 0),  (25, 0),   # Top row
    (0, 13), (13, 13), (25, 13),   # Middle row
    (0, 25), (13, 25), (25, 25),   # Bottom row
]
# Each viewport covers 15x15 cells
# Coverage per axis: 0-14, 13-27, 25-39 = 39/40 cells
# Overlap at columns/rows 13-14 and 25-27 gives double observations
```

Budget allocation:

- 9 viewports x 5 seeds = 45 queries (grid coverage)
- 5 remaining queries allocated to highest-uncertainty seeds

Entropy-Targeted Strategy (Multi-Sample)

Information-theoretic viewport selection:

```
1. Compute per-cell expected entropy from CalibrationModel (free)
   entropy_map = -sum(prior * log(prior))  # (40, 40)
   entropy_map[ocean] = 0  # static cells have zero information value
   entropy_map[mountain] = 0

2. Greedy viewport selection:
   for _ in range(n_viewports):
       score all (x, y) positions:
           region_entropy = sum(entropy_map[y:y+15, x:x+15] * ~covered)
           skip if >50% overlap with already-selected viewports
       select position with highest uncovered entropy
       mark region as covered

3. Query selected viewports (highest entropy first)
4. Repeat viewports with highest stochasticity (API randomness)
```

Data Structures Built from Observations

ObservationAccumulator

Per-cell class count accumulator across all viewport observations.

```
class ObservationAccumulator:
    counts: ndarray(H, W, 6)   # Observation class counts
    total: ndarray(H, W)       # Total observations per cell

    def add_viewport(viewport, grid):
        for row, col in grid:
            cls = terrain_to_class(grid[row][col])
            counts[y+row, x+col, cls] += 1
            total[y+row, x+col] += 1
```

GlobalMultipliers

Tracks observed vs expected class distributions across all seeds.

```
class GlobalMultipliers:
    observed: ndarray(6)   # Total observed class counts
    expected: ndarray(6)   # Total expected class counts (from prior)

    def add_observation(obs_class, prior):
        observed[obs_class] += 1
        expected += prior    # prior distribution for that cell

    def get_multipliers():
        ratio = observed / expected
        return ratio         # >1 = more than expected, <1 = less
```

FeatureKeyBuckets

Pools observations by feature key for empirical distributions.

```
class FeatureKeyBuckets:
    counts: dict[FK, ndarray(6)]   # Per-FK class counts
    totals: dict[FK, int]          # Per-FK observation count

    def add_observation(fk, obs_class):
        counts[fk][obs_class] += 1
        totals[fk] += 1

    def get_empirical(fk) -> (ndarray(6), int):
        return normalize(counts[fk]), totals[fk]
```

Statistics: ~11,250 observations pooled into ~120 FK buckets = ~94 observations per bucket (vs 1-2 per cell).

GlobalTransitionMatrix

Tracks terrain state transitions by distance and coastal status.

```
class GlobalTransitionMatrix:
    # dist_buckets: 0-1, 2-3, 4-5, 6-7, 8+
    # coastal variants tracked separately
    transitions: dict[(dist_bucket, coastal), ndarray(6)]
```

MultiSampleStore

Stores raw observation grids for variance analysis.

```
class MultiSampleStore:
    grids: list[ndarray(H, W)]    # Raw observation grids
    # Computes per-FK settlement variance
    # Detects stochastic API behavior (different results per query)
```

Observation Overlay Functions

`build_obs_overlay()`

Per-cell empirical distribution from direct observations.

```
def build_obs_overlay(observations, terrain, seed_idx):
    obs_counts = zeros(H, W, 6)  # Class counts per cell
    obs_total = zeros(H, W)      # Total observations per cell

    for obs in observations:
        if obs.seed_index != seed_idx: continue
        for row, col in obs.grid:
            cls = terrain_to_class(grid[row][col])
            obs_counts[y+row, x+col, cls] += 1
            obs_total[y+row, x+col] += 1

    return obs_counts, obs_total
```

Used in Dirichlet-Multinomial conjugate update (Stage 4.6 of prediction pipeline).

`build_sett_survival()`

Per-initial-settlement alive/dead evidence.

```
def build_sett_survival(observations, settlements, seed_idx):
    alive_counts = zeros(n_sett)  # Times observed alive
    dead_counts = zeros(n_sett)   # Times observed dead

    for obs in observations:
        if obs.seed_index != seed_idx: continue
        for cell in obs.grid:
            if cell_position in settlement_positions:
                if cls in (settlement, port): alive += 1
                else: dead += 1  # empty, ruin, forest = dead

    return alive_counts, dead_counts, (alive + dead) > 0
```

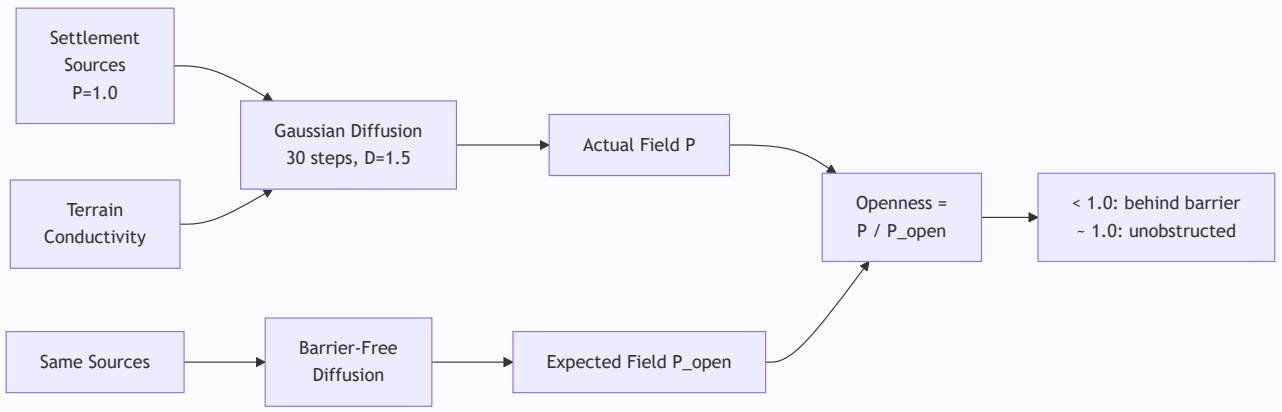
Strongest per-cell signal: binary alive/dead observed multiple times.

`build_growth_front_map()`

Heat map of active expansion fronts from young settlements.

```
def build_growth_front_map(observations, terrain):
    front_map = zeros(H, W)

    for settlement in obs.settlements:
        if settlement.population >= 1.0: continue  # Only young
        youth = 1.0 - population  # 0 (mature) to 1 (newborn)
        # Spread with Manhattan decay within r=3
        for dy, dx in manhattan_radius(3):
            front_map[sy+dy, sx+dx] += youth / (1 + manhattan_dist)

    return normalize_to_0_1(front_map)
```

`compute_terrain_openness()`



Diffusion-based terrain accessibility metric.

```
def compute_terrain_openness(terrain, settlements, n_steps=30, D=1.5, decay=0.12):
    # Initialize diffusion source at settlements
    P[settlement_cells] = 1.0

    # Diffusion with terrain conductivity
    conductivity[forest] = 0.7  # forests resist
    conductivity[ocean/mountain] = 0.0  # impassable

    for _ in range(n_steps):
        laplacian = sum(neighbor_conductivity * neighbor_P) - 4 * self_conductivity * P
        P += dt * (D * laplacian - decay * P)
        P[settlements] = 1.0  # source reset
        P[barriers] = 0.0     # hard boundary

    # Compare to barrier-free diffusion
    P_open = same_diffusion_but_uniform_conductivity()

    # Openness ratio: <1.0 = behind barrier, ~1.0 = unobstructed
    return clip(P / P_open, 0.0, 1.5)
```

Used in terrain barrier correction (autoloop `barrier_strength` parameter).

Astar Island Prediction System

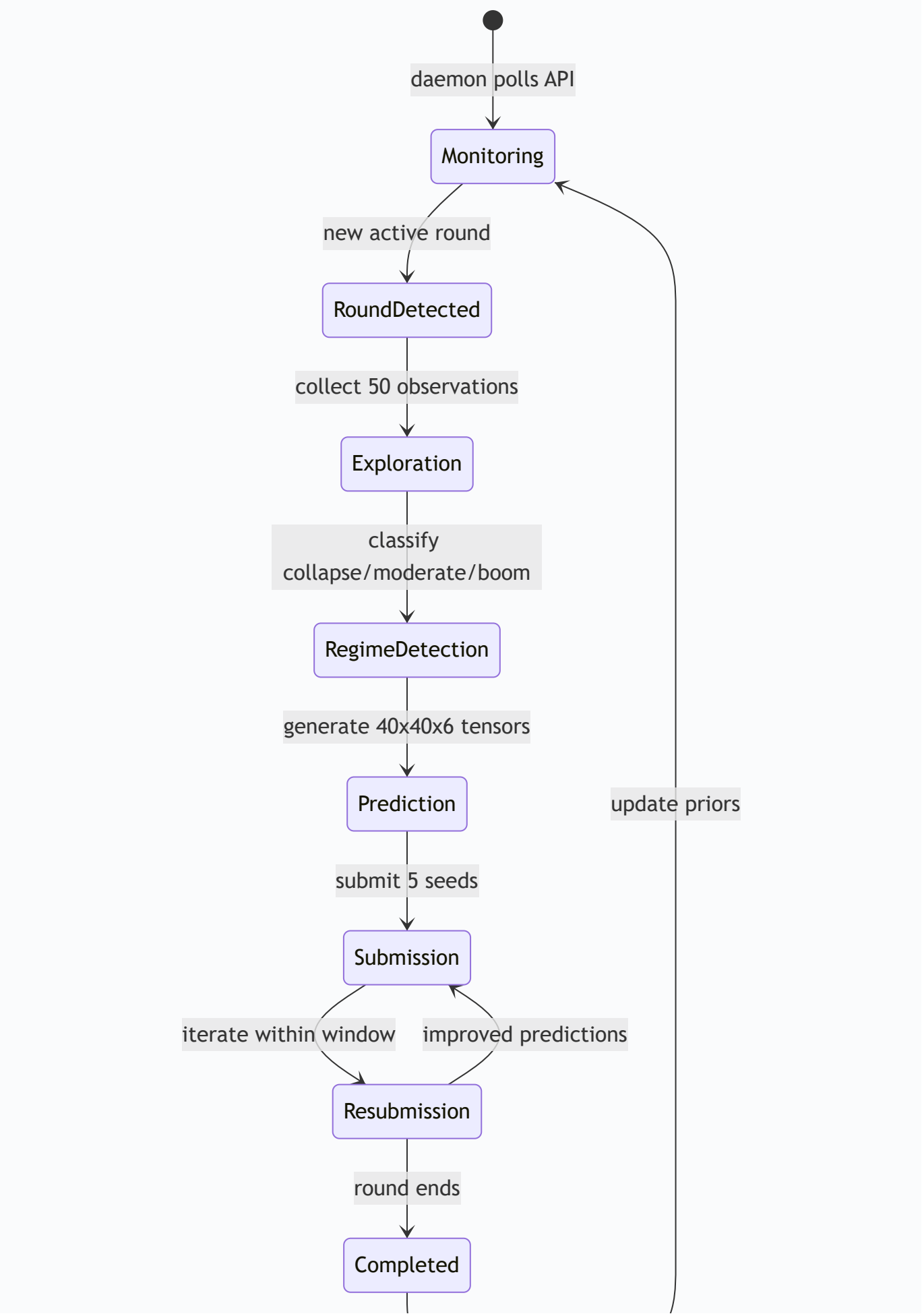
Autonomous system that observes a Norse civilization simulator, builds probabilistic models, and submits 40x40x6 probability tensor predictions across 5 seeds per round.

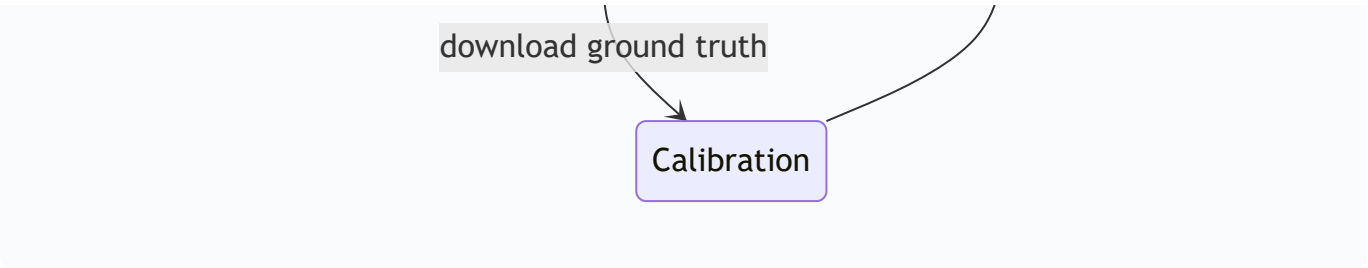
Features

- Adaptive exploration: entropy-targeted viewport selection within 50-query budget
- Hierarchical Bayesian calibration from 17 rounds of ground truth
- Regime detection (collapse/moderate/boom) from early observations
- Parametric simulator fitting via CMA-ES optimization
- Feature-key bucketing for empirical distribution pooling
- Continuous autonomous parameter optimization (770k+ experiments)
- Real-time web dashboard for monitoring all subsystems
- Automatic round detection and multi-seed submission

User Flows

Round Lifecycle


daemon polls API
Monitoring
new active round
RoundDetected
collect 50 observations
Exploration
classify
collapse/moderate/boom
RegimeDetection
generate 40x40x6 tensors
Prediction
update priors
submit 5 seeds
Submission
iterate within window
improved predictions
Resubmission
round ends
Completed



Subsystems

1. Daemon (daemon.py)

- Polls API every 60s for new rounds
- Downloads completed rounds' ground truth to `data/calibration/`
- Triggers exploration, prediction, and submission pipeline
- GPU resubmission loop for iterative improvement

2. Autoloop (autoloop_fast.py)

- Metropolis-Hastings parameter optimization
- Backtests against 8+ rounds x 5 seeds (leave-one-out)
- ~~70ms per experiment (~~160k experiments/hour)
- 40+ tunable parameters: FK blending, multipliers, temperature, calibration weights
- Greedy accept with 20% Metropolis exploration on near-misses

3. Prediction Engine (predict_gemini.py)

- Hierarchical calibration prior (fine/coarse/base/global)
- Empirical FK distribution blending
- Global multiplier adjustment (observed/expected ratios)
- Post-hoc corrections: growth fronts, survival evidence, terrain barriers
- Temperature scaling (entropy-aware)
- Optional simulator blend (alpha=0.20-0.35)

4. Simulator (sim_inference.py + sim_model_gpu.py)

- 17-parameter parametric simulator
- CMA-ES fitting with regime-specific warm starts
- KNN transfer learning from historical rounds

- GPU-accelerated: 10k simulations per evaluation

5. Web Dashboard (web/)

- SvelteKit app with 7 route pages
- Real-time: scores, autoloop progress, daemon status, logs
- Components: heatmaps, sparklines, gauge bars, flow diagrams

6. Multi-Researcher

- Claude Opus + Gemini Flash-Lite generate structural algorithm improvements
- Ideas saved as executable Python files in `data/multi_ideas/`
- Best ideas scored 86.7 (vs 89.4 baseline -- different evaluation scope)

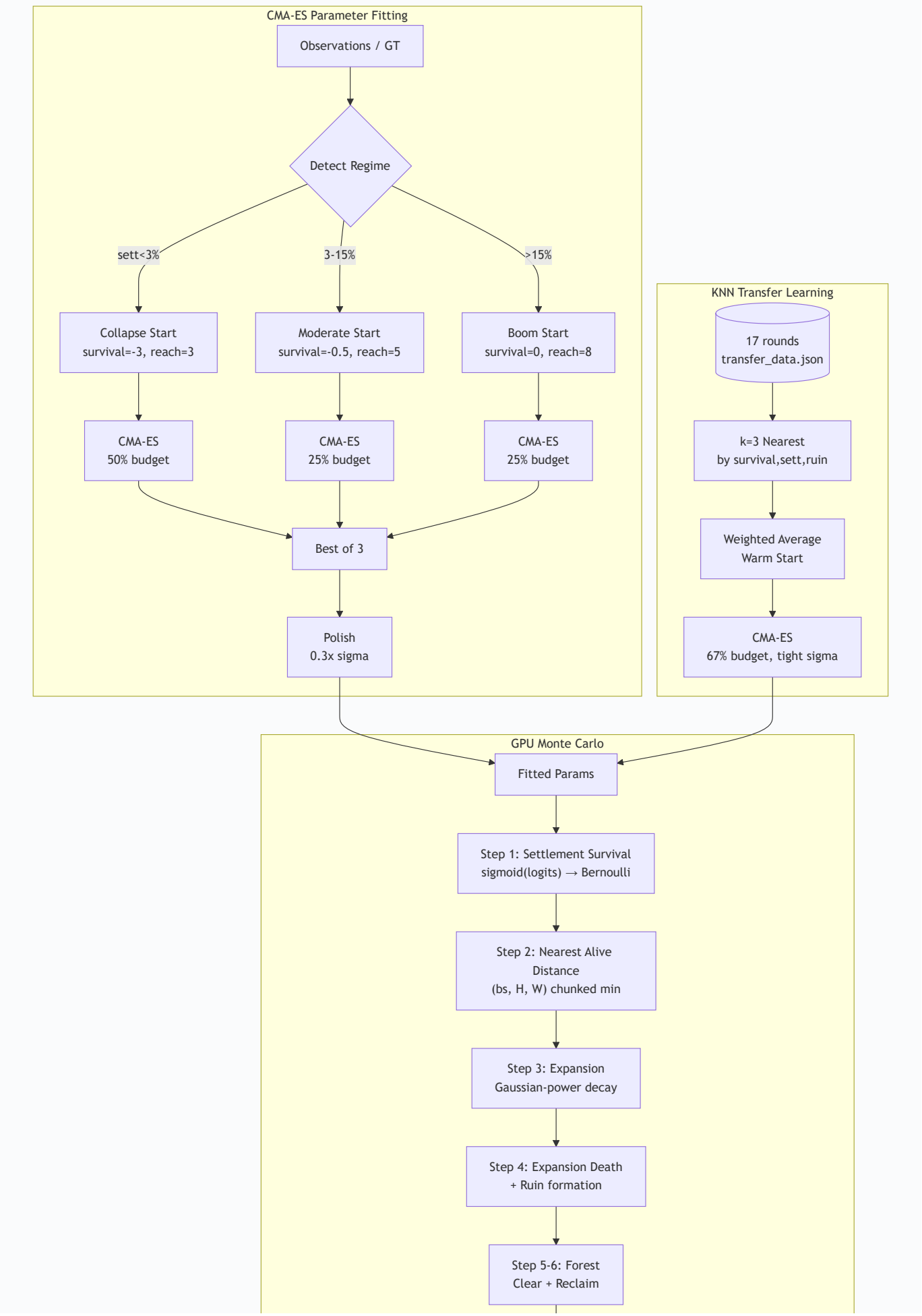
Edge Cases

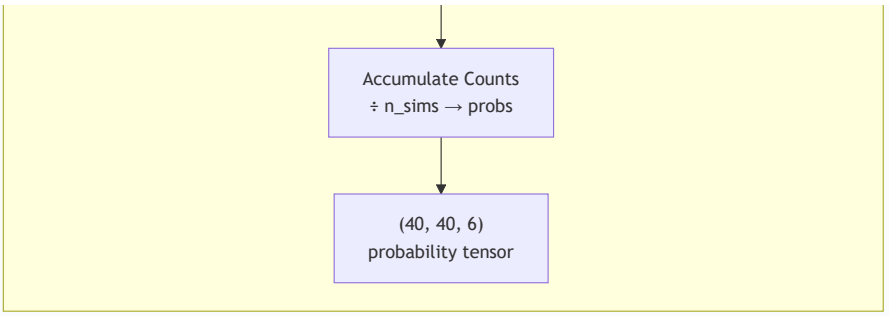
- Boom rounds (settlement rate >15%) score ~2 points lower than non-boom
- Collapse rounds (settlement <3%) require aggressive prior narrowing
- Stochastic API: same query returns different results across calls
- Rate limiting (429 responses): exponential backoff with retry

GPU Simulator -- Deep Technical Reference

PyTorch CUDA-accelerated Monte Carlo simulator for the Astar Island Norse civilization model. Drop-in replacement for the CPU numpy version with ~100x speedup.

Architecture


CMA-ES Parameter Fitting
Observations / GT
Detect Regime
sett<3%
3-15%
>15%
Collapse Start
survival=-3, reach=3
Moderate Start
survival=-0.5, reach=5
Boom Start
survival=0, reach=8
CMA-ES
50% budget
CMA-ES
25% budget
CMA-ES
25% budget
Best of 3
Polish
0.3x sigma
KNN Transfer Learning
17 rounds
transfer_data.json
k=3 Nearest
by survival,sett,ruin
Weighted Average
Warm Start
CMA-ES
67% budget, tight sigma
GPU Monte Carlo
Fitted Params
Step 1: Settlement Survival
sigmoid(logits) → Bernoulli
Step 2: Nearest Alive
Distance
(bs, H, W) chunked min
Step 3: Expansion
Gaussian-power decay
Step 4: Expansion Death
+ Ruin formation
Step 5-6: Forest
Clear + Reclaim




```
sim_model.py (CPU)          sim_model_gpu.py (GPU)
├── Simulator               ├── GPUSimulator
├── compute_score()         ├── fit_cma_gpu()
├── PARAM_SPEC (17 params)  └── (imports PARAM_SPEC from sim_model)
└── params_to_vec/vec_to_params

sim_inference.py
├── fit_to_gt()            # CMA-ES fitting against ground truth
├── fit_to_observations()  # CMA-ES fitting from live observations
├── WARM_STARTS            # 3 regime-specific starting points
└── _knn_warm_start()      # Transfer learning from historical rounds
```

Simulator Parameters (17 dimensions)

Defined in `sim_model.PARAM_SPEC`, each with (default, min, max):

Settlement Survival (5 params)

Parameter	Purpose	Typical Range
`base_survival`	Logit intercept for settlement survival	-3.0 to 0.0
`food_coeff`	Forest/food production coefficient	0.0 to 1.0
`coastal_mod`	Coastal settlement survival modifier	-1.0 to 0.0
`cluster_pen`	Cluster density linear penalty	-1.0 to 0.0
`cluster_optimal`	Optimal cluster density for inverted-U	1.0 to 5.0

Survival probability:

```
logits = base_survival
       + food_coeff * (food_within_r2 / 12.0)
       + coastal_mod * is_coastal
       + cluster_pen * (cluster_r3 / 3.0)
       + cluster_quad * ((cluster_r3 - cluster_optimal)^2 / 9.0)
surv_prob = sigmoid(logits)
```

Expansion (5 params)

Parameter	Purpose	Typical Range
`expansion_str`	Expansion probability coefficient	0.1 to 1.0
`expansion_scale`	Distance scale for expansion decay	1.0 to 6.0
`decay_power`	Power of distance decay (higher = sharper)	1.0 to 4.0
`max_reach`	Hard cutoff distance for expansion	2.0 to 10.0
`forest_resist`	Forest terrain expansion resistance	0.0 to 0.5

Expansion probability:

```
exp_prob = expansion_str * exp(-(d / expansion_scale)^decay_power)
exp_prob *= (1 - forest_resist) if cell_is_forest
exp_prob = 0 if d > max_reach
```

Post-Expansion (4 params)

Parameter	Purpose	Typical Range
`exp_death`	Probability expanded cell dies	0.1 to 0.8
`ruin_rate`	Probability dead cell becomes ruin	0.05 to 0.5
`port_factor`	Probability expansion on coast becomes port	0.05 to 0.5
`forest_clear`	Forest clearing rate near settlements	0.1 to 0.6

Environment (2 params)

Parameter	Purpose	Typical Range
`forest_reclaim`	Empty cell forest regrowth rate	0.01 to 0.15
`cluster_quad`	Quadratic cluster penalty coefficient	-0.5 to 0.0

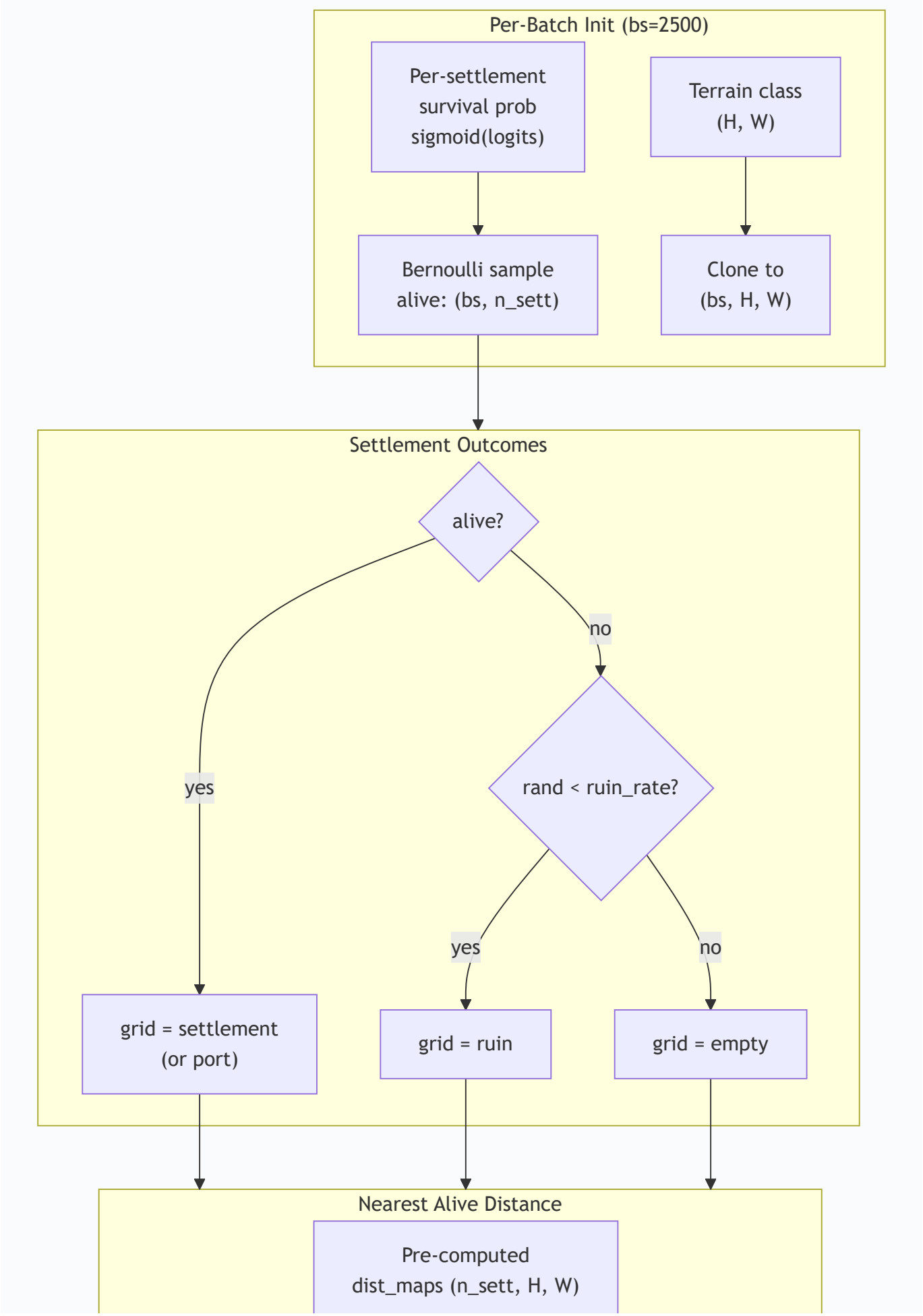
GPUSimulator Internals

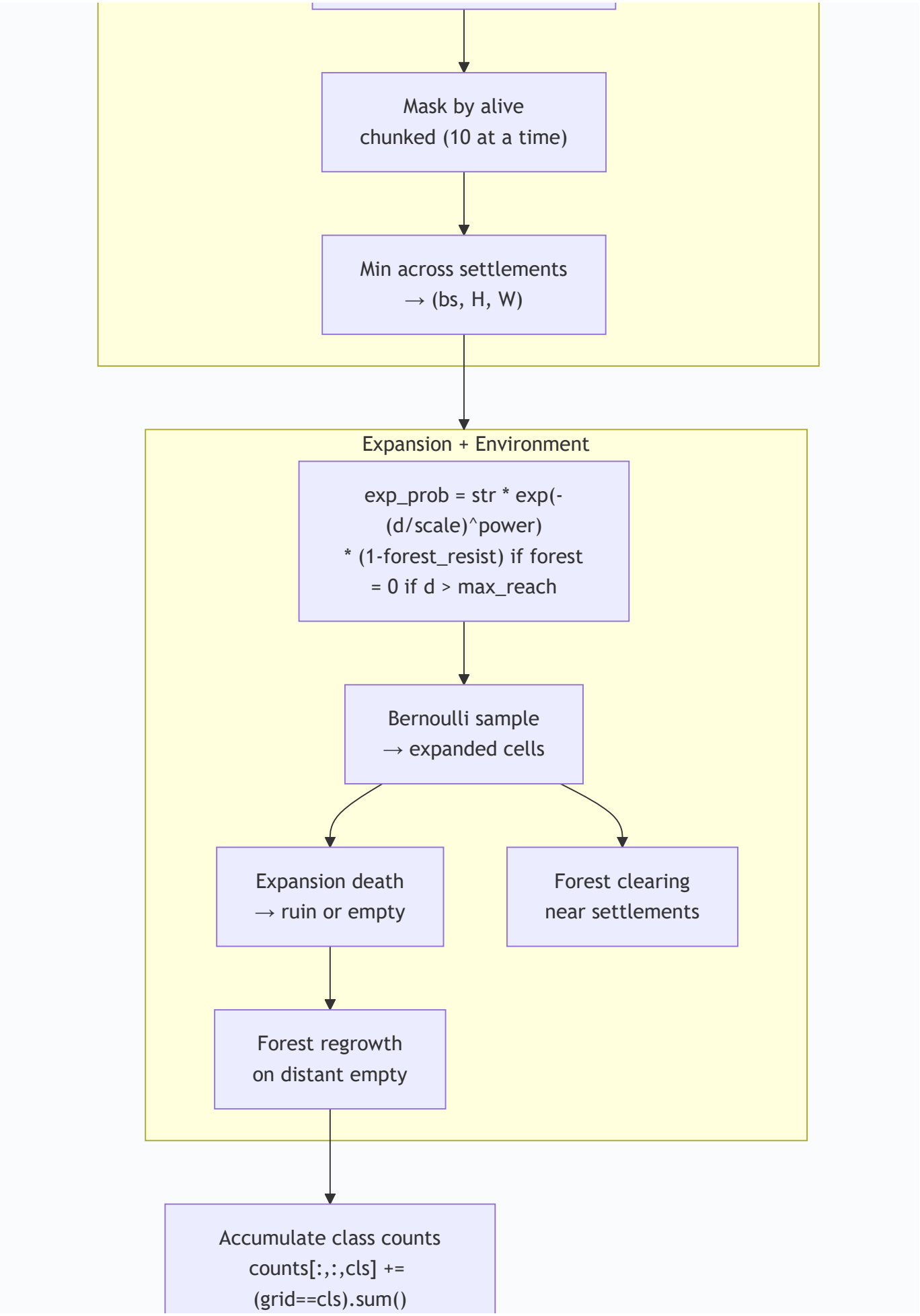
Initialization (`__init__`)

Pre-computes and transfers to CUDA:

1. **Terrain data** `(H, W)` as int8 tensor
2. **Terrain class mapping** `(H, W)` - raw codes to 6-class indices
3. **Settlement features:**
   - `sett_coastal` (n_sett,) float32 - is coastal flag
   - `sett_has_port` (n_sett,) bool - has port flag
   - `sett_cluster` (n_sett,) float32 - cluster density within r=3
   - `sett_food_r2` (n_sett,) float32 - food (forest/plains) within Manhattan r=2
4. **Distance maps** `(n_sett, H, W)` float32 - pre-computed per-settlement distances
5. **Cell masks:**
   - `cell_coastal` (H, W) bool
   - `cell_is_forest` (H, W) bool
   - `expand_mask` (H, W) bool - buildable and not already settlement

Simulation Loop (`run()`)


Per-Batch Init (bs=2500)
Per-settlement survival prob sigmoid(logits)
Terrain class (H, W)
Bernoulli sample alive: (bs, n_sett)
Clone to (bs, H, W)
Settlement Outcomes
alive?
yes
no
rand < ruin_rate?
yes
no
grid = settlement (or port)
grid = ruin
grid = empty
Nearest Alive Distance
Pre-computed dist_maps (n_sett, H, W)


Mask by alive
chunked (10 at a time)
Min across settlements
→ (bs, H, W)
Expansion + Environment
exp_prob = str * exp(-(d/scale)^power)
* (1-forest_resist) if forest
= 0 if d > max_reach
Bernoulli sample
→ expanded cells
Expansion death
→ ruin or empty
Forest clearing
near settlements
Forest regrowth
on distant empty
Accumulate class counts
counts[:,:,cls] +=
(grid==cls).sum()

Batched execution: processes `batch_size` simulations in parallel (default 2500).

For each batch of `bs` simulations:

```
probs = counts / n_sims
→ (H, W, 6)
```

```
Step 1: Sample settlement survival
    alive = rand(bs, n_sett) < surv_prob   # Bernoulli sampling

Step 2: Initialize grids from terrain class
    grids = terrain_class.expand(bs, H, W)  # (bs, H, W)

Step 3: Apply settlement outcomes
    for each settlement:
        if alive: grid[sy,sx] = settlement (or port if has_port)
        if dead and rand < ruin_rate: grid[sy,sx] = ruin
        if dead and no ruin: grid[sy,sx] = empty

Step 4: Compute nearest alive distance
    # (bs, H, W) - minimum distance to any alive settlement
    # Processed in chunks of 10 settlements to balance memory
    nearest = min over settlements(dist_map * alive_mask)

Step 5: Compute expansion probability
    exp_prob = expansion_str * exp(-(nearest/scale)^power)
    exp_prob *= (1 - forest_resist) where forest
    exp_prob = 0 where nearest > max_reach
    exp_prob = 0 where not buildable

Step 6: Sample and apply expansion
    expanded = rand(bs, H, W) < exp_prob
    port_expanded = expanded & coastal & rand < port_factor
    sett_expanded = expanded & ~port_expanded

Step 7: Expansion death
    exp_dies = expanded & rand < exp_death
    exp_ruin = exp_dies & rand < ruin_rate

Step 8: Forest clearing
    clear_prob = forest_clear * exp(-(nearest/3)^2)
    cleared = is_forest & rand < clear_prob

Step 9: Forest reclamation
    reclaim_prob = base_rate * min(nearest/5, 1)
    # Higher regrowth on originally-forested cells
    reclaimed = is_empty & rand < reclaim_prob

Step 10: Accumulate counts
    for class in 0..5:
        counts[:,:,class] += (grids == class).sum(dim=0)
```

Output: `probs = counts / n_sims` -> `(H, W, 6)` numpy array

Performance

Setting	CPU (sim_model)	GPU (sim_model_gpu)
500 sims	~0.5s	~0.05s
5000 sims	~5s	~0.3s
10000 sims	~10s	~0.5s
Memory (L4 24GB)	N/A	~2 GB per run

CMA-ES Fitting Pipeline

`fit_to_gt()` -- Fitting Against Ground Truth

Used for offline calibration of all historical rounds.

```
Input: RoundData (terrain, settlements, ground_truth)

1. Detect regime from GT: collapse (<3% sett), boom (>15%), moderate
2. Multi-start CMA-ES:
   a. Detected regime start (50% of budget)
   b. Regime 2 start (25% of budget)
   c. Regime 3 start (25% of budget)
3. Polish: refine best with 0.3x sigma (25% of budget)

Objective: minimize -compute_score(gt, sim.run(params))
CMA-ES options:
  - bounds: PARAM_SPEC min/max
  - tolfun: 0.01 (stop if improvement < 0.01)
  - tolx: 1e-4 (stop if step < 1e-4)

Output: (best_params, best_score)
```

`fit_to_observations()` -- Fitting From Live Observations

Used during live rounds when no ground truth is available.

```
Input: RoundData, observations (from API)

1. Compute observable features from observations:
   - survival_rate: initial settlements still alive / total observed
   - sett_rate: settlement class / total dynamic cells observed
   - ruin_rate: ruin class / total dynamic cells observed

2. KNN warm-start from transfer_data.json:
   - Load GT-fitted params from all 17 historical rounds
   - Find k=3 nearest neighbors by Euclidean distance in feature space
   - Average their params weighted by 1/(distance + 0.01)

3. CMA-ES optimization:
   a. KNN warm start (67% budget, sigma=0.2, tighter)
   b. Regime warm start (33% budget, sigma=0.5, wider)
   Pick best result.

Objective: maximize mean log-likelihood of observed cells
   pred = sim.run(params)
   ll = mean(log(pred[obs_y, obs_x, obs_class]))

Output: (best_params, best_score_on_gt_if_available)
```

Transfer Learning Data

`data/sim_params/transfer_data.json` stores GT-fitted params and features for every historical round:

```
[
  {
    "round": "round5",
    "params": {"base_survival": -0.47, "expansion_str": 0.52, ...},
    "features": {
      "gt_survival": 0.65,
      "gt_sett_rate": 0.08,
      "gt_ruin_rate": 0.02
    },
    "score": 82.5
  },
  ...
]
```

Regime-Specific Warm Starts

Three pre-computed starting points learned from fitting 15+ rounds:

Collapse (settlement <3%)

```
base_survival=-3.0, expansion_str=0.3, expansion_scale=1.5,
decay_power=3.0, max_reach=3.0, exp_death=0.7
```

Key: very low survival, narrow expansion, high death rate.

Moderate (3-15% settlement)

```
base_survival=-0.5, expansion_str=0.5, expansion_scale=2.5,
decay_power=2.5, max_reach=5.0, exp_death=0.4
```

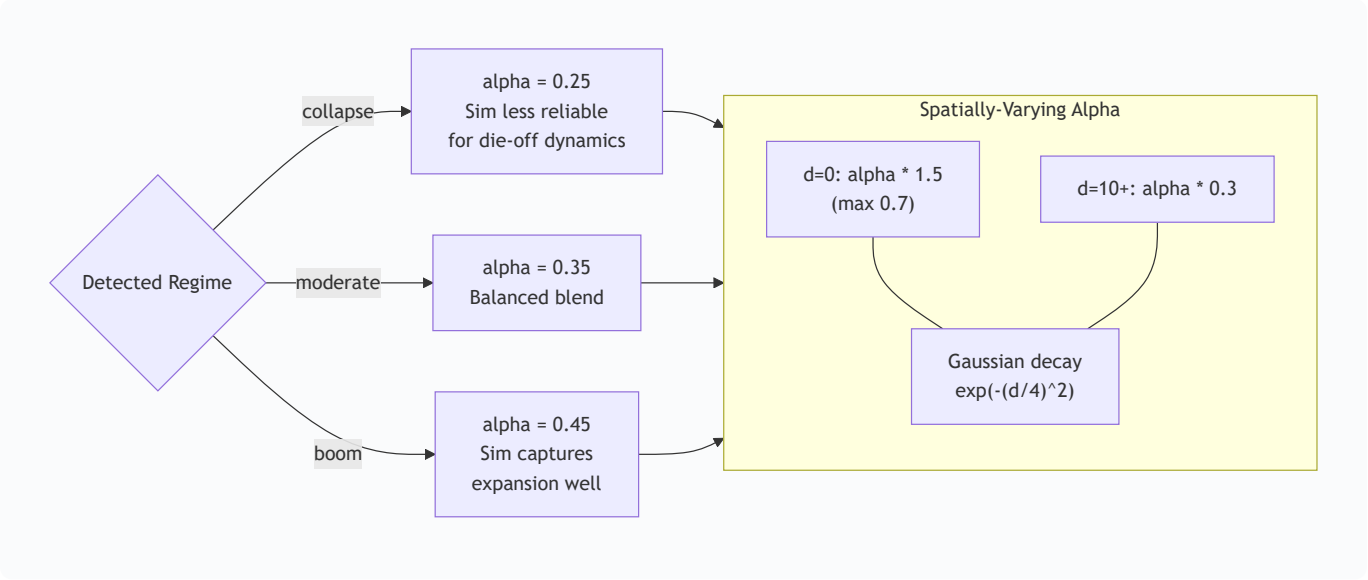
Key: balanced survival and expansion.

Boom (settlement >15%)

```
base_survival=0.0, expansion_str=0.8, expansion_scale=4.0,
decay_power=1.5, max_reach=8.0, exp_death=0.3
```

Key: high survival, wide expansion, low death.

Adaptive Alpha (Simulator Blend Weight)



The blend weight between statistical model and simulator varies by regime:

```
REGIME_ALPHAS = {
    "collapse": 0.25,  # Simulator less reliable for collapse dynamics
    "moderate": 0.35,  # Balanced blend
    "boom": 0.45,      # Simulator captures expansion well in boom
}
```

In `predict_gemini.py`, the blend is spatially varying:

```
alpha_near = min(sim_alpha * 1.5, 0.7)   # Higher weight near settlements
alpha_far = sim_alpha * 0.3               # Lower weight far from settlements
alpha_map = alpha_far + (alpha_near - alpha_far) * exp(-(dist/4)^2)
pred = (1 - alpha_map) * statistical_pred + alpha_map * sim_pred
```

`fit_cma_gpu()` -- GPU-Optimized CMA-ES

Dedicated GPU fitting function with normalized parameter space:

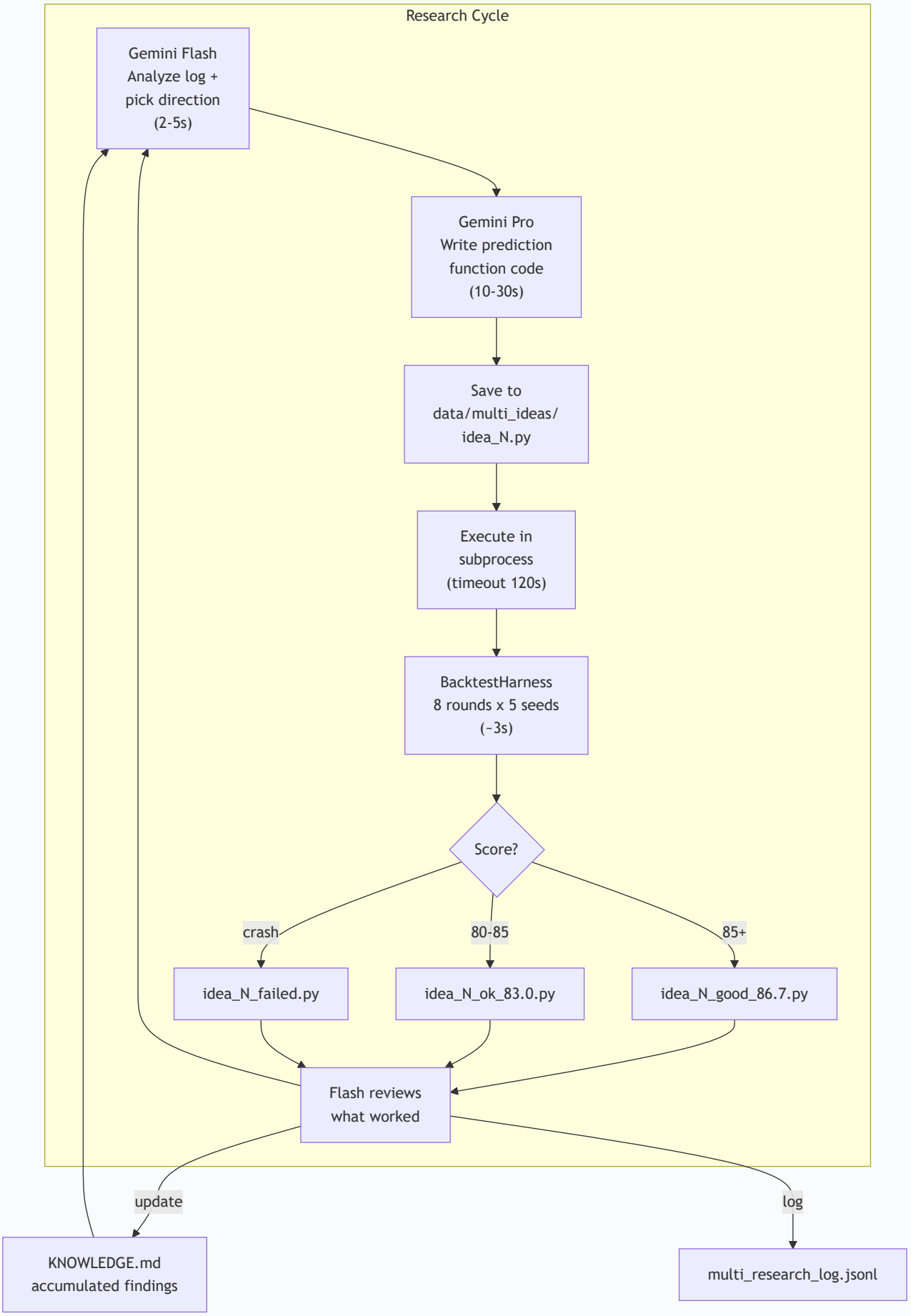
```
# Normalize parameters to [0,1] for uniform step sizes
x_norm = (x - lo) / (hi - lo)

# CMA-ES in normalized space
es = CMA(x0_norm, sigma=0.5, bounds=[0, 1])

# Each evaluation:
x = lo + x_norm * (hi - lo)  # denormalize
pred = gpu_sim.run(params, n_sims=1000)
score = compute_score(target, pred)
```

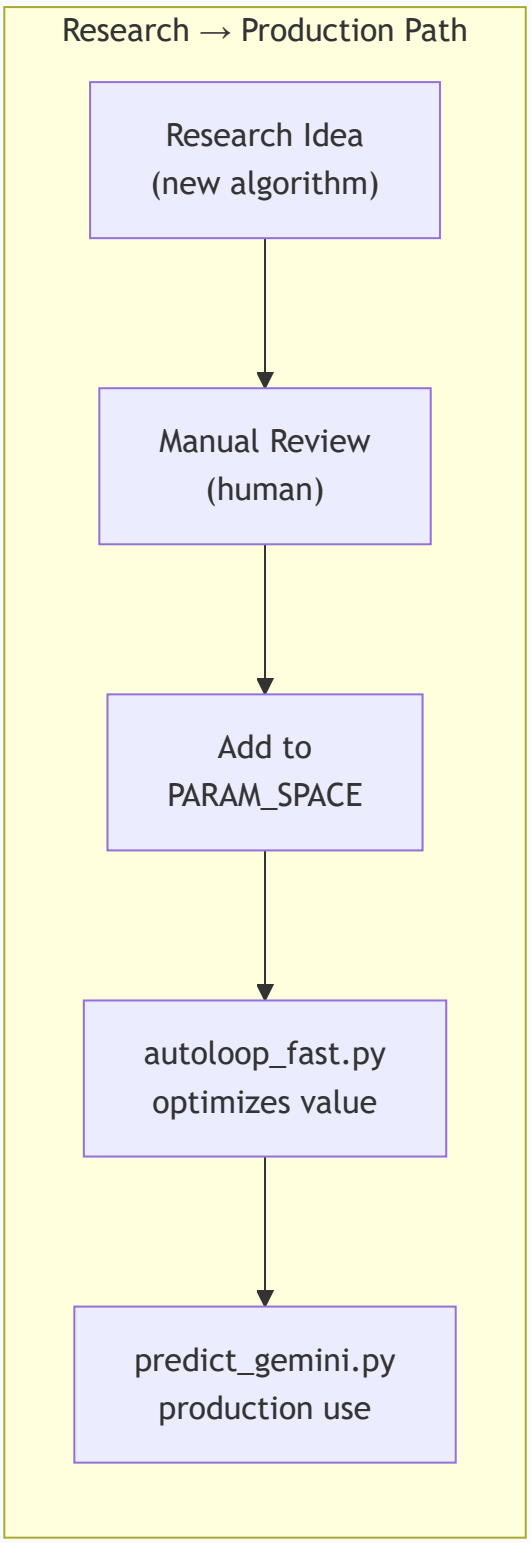
Used by `daemon.py:gpu_resubmit_round()` for iterative improvement during active rounds.

Multi-Researcher -- Technical Reference

Autonomous research agent that uses LLMs to generate, evaluate, and iterate on structural algorithm improvements for the prediction pipeline.

Architecture


Research Cycle
Gemini Flash
Analyze log +
pick direction
(2-5s)
Gemini Pro
Write prediction
function code
(10-30s)
Save to
data/multi_ideas/
idea_N.py
Execute in
subprocess
(timeout 120s)
BacktestHarness
8 rounds x 5 seeds
(~3s)
Score?
crash
80-85
85+
idea_N_failed.py
idea_N_ok_83.0.py
idea_N_good_86.7.py
Flash reviews
what worked
update
log
KNOWLEDGE.md
accumulated findings
multi_research_log.jsonl


Research → Production Path
Research Idea
(new algorithm)
Manual Review
(human)
Add to
PARAM_SPACE
autoloop_fast.py
optimizes value
predict_gemini.py
production use

Models Used

Model	Role	Latency	Cost
Gemini 3.1 Pro Preview	Code generation (replaces Claude Opus)	10-30s	Higher
Gemini 3 Flash Preview	Fast analysis, direction picking (replaces Claude Haiku)	2-5s	Lower

Both accessed via `google-genai` SDK with `GOOGLE_API_KEY` from `.env`.

Research Cycle

Step 1: Direction Selection (Flash)

Flash receives:

- Current best score and parameters
- Last 5-10 experiment results (accepted + rejected)
- Knowledge base (`KNOWLEDGE.md`) of accumulated findings
- Available parameter space and current values

Flash outputs:

- 1-3 promising research directions
- Rationale for each
- Specific parameters/code areas to modify

Step 2: Code Generation (Pro)

Pro receives:

- Selected direction from Flash
- Current prediction function source code
- Full parameter space definition
- Calibration model interface
- Examples of previous successful modifications

Pro outputs:

- Complete modified prediction function

- Wrapped in executable Python that matches BacktestHarness interface

Step 3: Evaluation

```
# Extract Python code from LLM response
code = extract_code_block(response)

# Write to data/multi_ideas/idea_{N}.py
idea_path = save_idea(code, idea_number)

# Execute in subprocess for safety
result = subprocess.run(["python", str(idea_path)], capture_output=True, timeout=120)

# Parse score from stdout
score = extract_score(result.stdout)
```

Step 4: Analysis (Flash)

Flash receives:

- Generated code and its score
- How it compares to current best
- Per-round breakdown (boom vs non-boom)

Flash decides:

- Whether this direction is promising (continue iterating)
- What to modify next (narrow search)
- Or abandon direction (try something else)

Idea Files

Generated algorithms saved to `data/multi_ideas/idea_{N}_{status}.py`:

Status	Meaning	Example
`ok_83.0`	Ran successfully, scored 83.0	`idea_0001_ok_83.0.py`
`good_86.7`	Scored above threshold (85+)	`idea_0076_good_85.7.py`
`failed`	Crashed, timeout, or invalid output	`idea_0002_failed.py`

Current Statistics (as of 2026-03-21)

- Total ideas: 166+
- Successful (ok): ~60
- Good (85+): ~8
- Failed: ~40
- Best score: 86.7 (idea_0153, idea_0155)

Note: Research ideas are evaluated on a different harness configuration than autoloop, so scores are not directly comparable. The best research score (86.7) doesn't beat autoloop's 89.4 because research ideas don't use the full production pipeline (observation overlays, survival evidence, etc.).

Knowledge Base

`KNOWLEDGE.md` accumulates findings across all research iterations:

- Which directions improved scores
- Which directions are dead ends
- Key insights about the scoring function
- Discovered parameter interactions
- Structural improvements that worked

Integration with Autoloop

Research ideas that show improvement are manually reviewed and integrated:

1. Researcher generates structural improvement (new feature/algorithm)
2. If score improves: extract the innovation
3. Add as new parameter to `PARAM_SPACE` in `autoloop.py`
4. Add implementation to `autoloop_fast.py:FastHarness.evaluate()`
5. Add production implementation to `predict_gemini.py`
6. Autoloop optimizes the new parameter's value

Logging

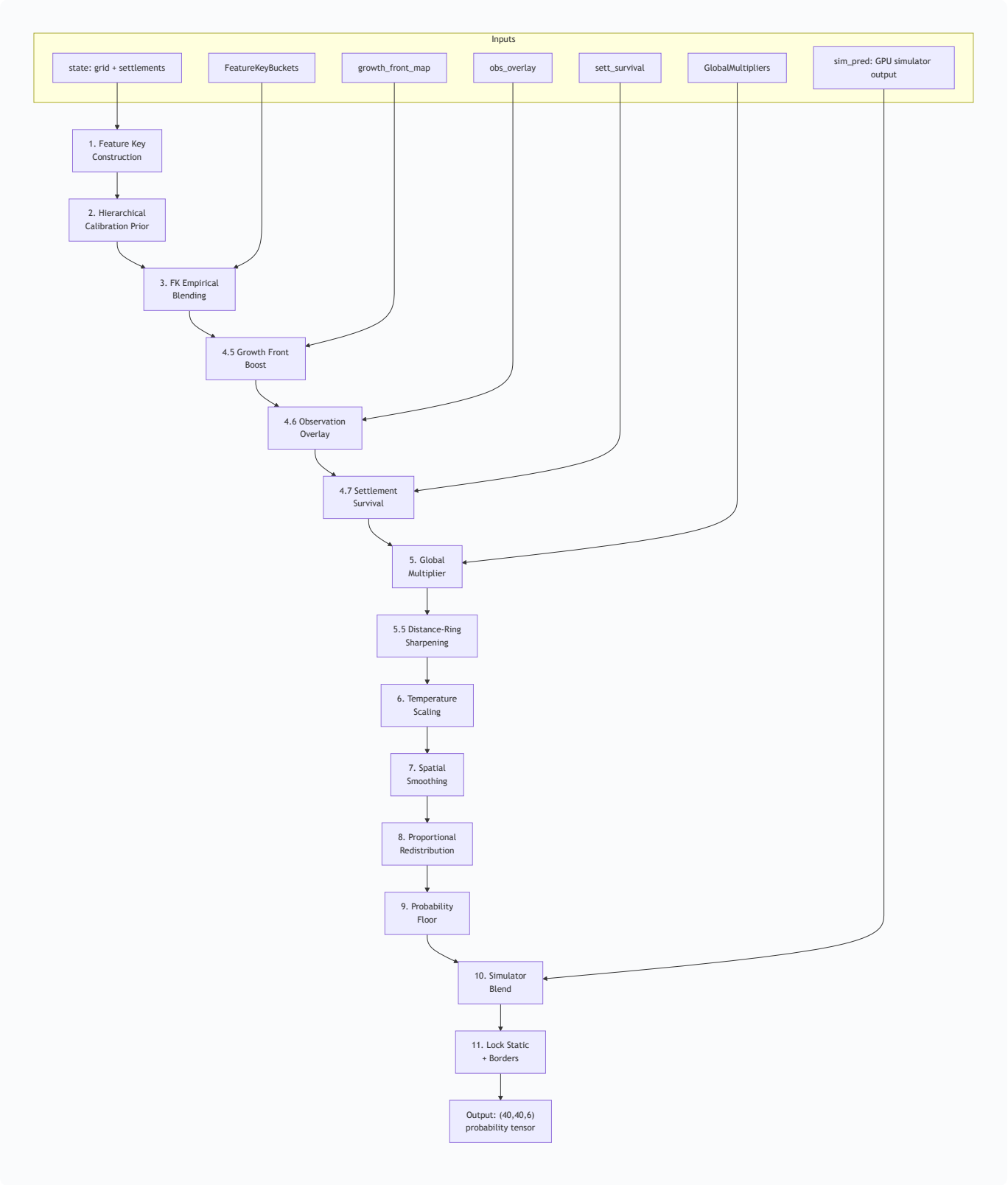
`data/multi_research_log.jsonl` tracks every research iteration:

```
{
    "id": 153,
    "direction": "entropy-aware temperature with settlement proximity boost",
    "model": "gemini-3.1-pro-preview",
    "code_file": "idea_0153_good_86.7.py",
    "score": 86.7,
    "per_round": {"round2": 88.1, "round3": 85.2, ...},
    "accepted": true,
    "timestamp": "2026-03-21T18:30:00Z"
}
```

Prediction Pipeline -- Step-by-Step Technical Reference

Complete walkthrough of `predict_gemini.py:gemini_predict()` -- the 11-stage production prediction engine.

Pipeline Overview



Input

```
gemini_predict(
    state: dict,           # {"grid": [[int]], "settlements": [{"x","y","has_port"}]}
    global_mult,           # GlobalMultipliers instance
    fk_buckets,           # FeatureKeyBuckets instance
    multi_store=None,     # MultiSampleStore (unused in current best)
    variance_regime=None, # "EXTREME_BOOM" or None
    obs_expansion_radius=None,  # dict {dist: (sett_count, total)} or int
    est_vigor=None,       # float: estimated settlement % on dynamic cells
    sim_pred=None,        # (40,40,6) numpy array from GPU simulator
    sim_alpha=0.25,       # blend weight for simulator predictions
    growth_front_map=None,  # (40,40) float: young settlement expansion activity
    obs_overlay=None,     # (obs_counts, obs_total) per-cell observation data
    sett_survival=None,   # (alive_counts, dead_counts, observed) per settlement
) -> np.ndarray  # (40, 40, 6) probability tensor
```

Stage 1: Feature Key Construction

```
fkeys = build_feature_keys(grid, settlements)
# Returns (40, 40) grid of 6-tuples:
# (terrain_code, dist_bucket, coastal, forest_neighbors, has_port_flag, cluster_bucket)
```

Distance buckets: 0, 1, 2, 3, 4-5, 6-8, 9+
Cluster buckets: 0 (isolated), 1 (sparse, 1-2), 2 (dense, 3+)

```
idx_grid, unique_keys = _build_feature_key_index(fkeys)
# idx_grid: (40, 40) int array -> index into unique_keys
# unique_keys: list of unique feature tuples (typically ~120 unique)
```

Stage 2: Hierarchical Calibration Prior

If `est_vigor` is provided, uses regime-conditional calibration:

```
cal = predict.get_regime_calibration(est_vigor)
# Weights training rounds by Gaussian similarity to estimated vigor
```

Otherwise uses uniform calibration from all historical rounds.

Lookup table construction (`build_calibration_lookup`):

For each unique feature key:

```
fine_weight = min(cal_fine_max, cal_fine_base + fine_count / cal_fine_divisor)
coarse_weight = min(cal_coarse_max, cal_coarse_base + coarse_count / cal_coarse_divisor)
base_weight = min(cal_base_max, cal_base_base + base_count / cal_base_divisor)
global_weight = cal_global_weight (fixed)

prior = (fine_weight * fine_dist + coarse_weight * coarse_dist
       + base_weight * base_dist + global_weight * global_dist)
prior /= sum(weights)
```

Output: `(N_unique, 6)` lookup table indexed by `idx_grid` .

Stage 3: FK Empirical Blending

Builds empirical distributions from observation-derived FK bucket data:

```
empiricals, counts = build_fk_empirical_lookup(fk_buckets, unique_keys, min_count=5)
# Only FKs with >= 5 observations get empirical data

strengths = min(emp_max, sqrt(counts))  # default: sqrt scaling
blended = pred * prior_w + empirical * strength
blended /= sum(blended)

pred = where(has_fk, blended, pred)  # only blend where sufficient data
```

EXTREME_BOOM regime adjustment:

```
prior_w -= 0.5    # Trust observations more in extreme boom
emp_max *= 1.2    # Allow stronger empirical signal
```

Stage 4.5: Growth Front Boost

Identifies cells near young, actively expanding settlements:

```
gf_factor = 1.0 + growth_front_boost * growth_front_map
pred[:, :, 1] *= gf_factor  # boost settlement probability
pred /= sum(pred)  # renormalize
```

`growth_front_map` is built from observation data: cells where settlements have population < 1.0 (young) get Manhattan-decay influence within r=3.

Stage 4.6: Observation Overlay (Dirichlet-Multinomial)

Conjugate Bayesian update on directly observed cells:

```
# obs_counts: (40, 40, 6) - observed class counts per cell
# obs_total: (40, 40) - total observations per cell
# obs_pseudo: model weight (higher = trust model more)

pred[observed] = (obs_pseudo * pred[observed] + obs_counts[observed]) / (obs_pseudo + obs_t
```

Interpretation: With `obs_pseudo=50` and `obs_total=5`:

- Model contributes 50/55 = 91% weight
- Observations contribute 5/55 = 9% weight

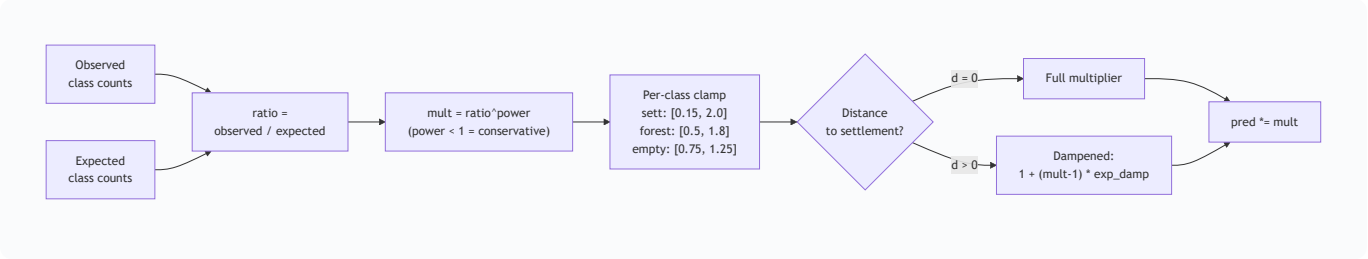
Stage 4.7: Settlement Survival Correction

Per-initial-settlement Bayesian update based on alive/dead evidence:

```
for each initial settlement (sy, sx):
    if observed:
        sett_counts = zeros(6)
        sett_counts[alive_cls] = alive_count       # alive -> settlement (or port)
        sett_counts[0] = dead_count * 0.5           # dead -> 50% empty
        sett_counts[3] = dead_count * 0.5           # dead -> 50% ruin

        pred[sy, sx] = (sett_pseudo * pred[sy, sx] + sett_counts) / (sett_pseudo + n_obs)
```

Key insight: Settlement cells have the strongest per-cell signal because alive/dead is binary and observed multiple times across queries.

Stage 5: Global Multiplier



Adjusts predictions based on observed-vs-expected class distributions.

```
ratio = observed / expected  # per-class ratio from GlobalMultipliers
mult = ratio^power           # dampened (power < 1 = conservative)
mult[1] = ratio[1]^sett_power  # settlement-specific power
mult[2] = ratio[2]^port_power  # port-specific power
mult[class] = clip(mult[class], lo, hi)  # per-class clamping
```

Distance-aware application:

```
# Settlements (dist=0): full multiplier
pred[dist==0] *= mult

# Expansion zone (dist>0): dampened multiplier
mult_exp[c] = 1.0 + (mult[c] - 1.0) * exp_damp
pred[dist>0] *= mult_exp
```

Stage 5.5: Distance-Ring Sharpening

Corrects the spatial profile of settlement predictions using per-distance observed rates:

```
for distance d in 0..12:
    if observed_total[d] < 30: continue  # need statistical significance
    obs_rate = observed_sett[d] / observed_total[d]
    pred_rate = mean(pred[dist==d, settlement_class])
    correction = clip(obs_rate / pred_rate, 0.2, 5.0)
    adjustment = 1.0 + dist_sharpen * (correction - 1.0)
    pred[dist==d, settlement_class] *= adjustment
```

Purpose: If observations show settlements concentrated at d=1-3 but the model predicts d=5+, this corrects the spatial distribution.

Stage 6: Entropy-Weighted Temperature

```
cal_entropy = -sum(cal_prior * log(cal_prior))  # per-cell entropy from calibration
t_frac = clip((cal_entropy - T_ent_lo) / (T_ent_hi - T_ent_lo), 0, 1)
T = T_low + t_frac * (T_high - T_low)

# Near-settlement boom boost
boom_boost = 0.10 * sqrt(min(ratio[settlement], 1.0))
T[dist <= sett_radius] += boom_boost

pred = pred^(1/T)  # T>1 softens (high entropy), T<1 sharpens (low entropy)
```

Stage 7: Selective Spatial Smoothing

3x3 uniform filter on settlement and ruin channels only:

```
for class in [settlement, ruin]:
    smoothed = uniform_filter(pred[:,:,class], size=3)
    pred[:,:,class] = pred[:,:,class] * (1 - smooth_alpha) + smoothed * smooth_alpha
```

Stage 8: Proportional Redistribution

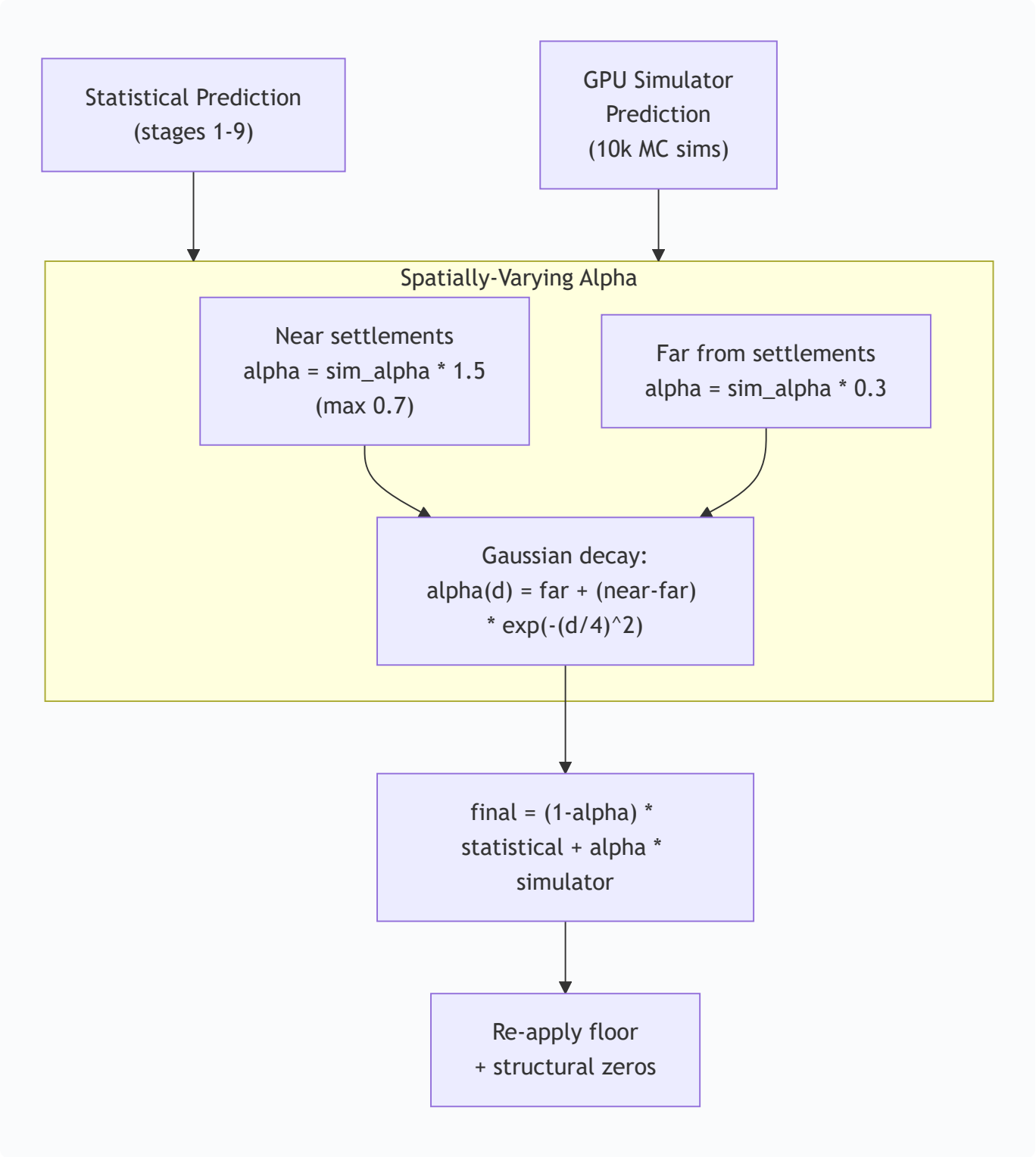
Removes probability mass from impossible classes and redistributes:

```
# Mountain mass on non-mountain cells
mountain_mass = pred[non_mountain, mountain_class]
pred[non_mountain, mountain_class] = 0

# Port mass on non-coastal cells
port_mass = pred[inland, port_class]
pred[inland, port_class] = 0

# Redistribute freed mass proportionally to calibration prior
redist_weights = cal_prior (excluding zeroed classes)
pred += freed_mass * redist_weights
```

Stage 9: Probability Floor

```
pred = max(pred, floor)           # default floor = 0.008
pred[non_mountain, mountain] = 0  # re-zero structural impossibilities
pred[non_coastal, port] = 0
pred /= sum(pred)                 # renormalize
```

Stage 10: Simulator Ensemble Blend



Spatially-varying blend with parametric simulator predictions:

```
alpha_near = min(sim_alpha * 1.5, 0.7)
alpha_far = sim_alpha * 0.3
alpha_map = alpha_far + (alpha_near - alpha_far) * exp(-(dist/4)^2)

pred = (1 - alpha_map) * pred + alpha_map * sim_pred
# Re-apply floor and structural zeros after blending
```

Stage 11: Lock Static + Borders

```
pred[ocean] = [1, 0, 0, 0, 0, 0]
pred[mountain] = [0, 0, 0, 0, 0, 1]
pred[border_rows_and_cols] = [1, 0, 0, 0, 0, 0]  # borders always empty
```

Output

`(40, 40, 6)` numpy float64 array where each cell sums to 1.0. Classes:

- 0: Empty (ocean, plains, cleared land)
- 1: Settlement
- 2: Port
- 3: Ruin
- 4: Forest
- 5: Mountain

Astar Island - Technical Design

Three-stage prediction pipeline: exploration, modeling, and submission, backed by continuous autonomous optimization.

Core Data Models

Prediction Tensor

```
prediction: np.ndarray  # shape (40, 40, 6), dtype float64
# Classes: [empty, settlement, port, ruin, forest, mountain]
# Constraints: sum per cell = 1.0, min per class = 0.01
```

Feature Key (per cell)

```
feature_key = (
    terrain_code,       # int: terrain type at cell
    dist_bucket,        # int: distance to nearest initial settlement
    coastal,            # bool: adjacent to ocean
    forest_neighbors,   # int: count of adjacent forest cells
    has_port,           # bool: initial port present
    cluster_bucket,     # int: settlement cluster density bucket
)
```

Calibration Model

```
class CalibrationModel:
    fine: dict[tuple, np.ndarray]    # 6-tuple FK -> class distribution
    coarse: dict[tuple, np.ndarray]  # 4-tuple FK -> class distribution
    base: dict[tuple, np.ndarray]    # 1-tuple (terrain,) -> class distribution
    global_: np.ndarray              # overall distribution
```

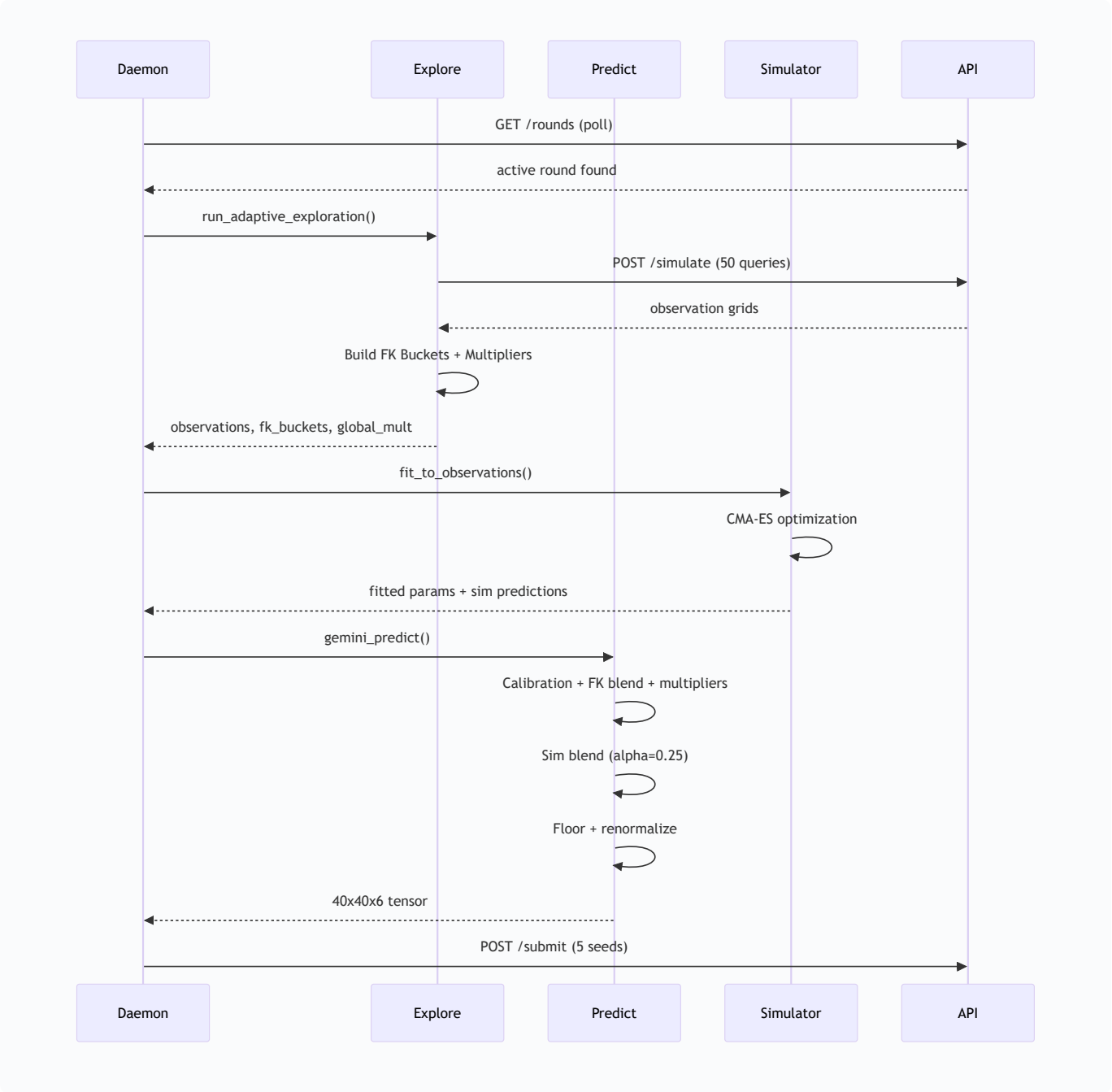
Simulator Parameters (17 dimensions)

```
params = {
    "base_survival", "expansion_str", "expansion_scale",
    "decay_power", "max_reach", "coastal_mod",
    "food_coeff", "cluster_pen", "cluster_optimal", "cluster_quad",
    "ruin_rate", "port_factor", "forest_resist",
    "forest_clear", "forest_reclaim", "exp_death"
}
```

API Endpoints

Method	Path	Description
GET	`/rounds`	List all rounds
GET	`/rounds/{id}`	Round detail + initial states
POST	`/rounds/{id}/simulate`	Query 15x15 viewport (costs 1 query)
POST	`/rounds/{id}/submit`	Submit 40x40x6 prediction for a seed

Prediction Pipeline



Autoloop Optimization

Parameter Space (40+ dimensions)

Group	Parameters	Examples
FK Blending	4	prior_weight, max_strength, min_count, strength_fn
Multipliers	10	power, per-class bounds, dampening
Temperature	4	T_low, T_high, entropy thresholds
Spatial	2	smooth_alpha, barrier_strength
Calibration	9	fine/coarse/base weights and divisors
Growth/Obs	3	growth_front_boost, obs_overlay_alpha

Optimization Algorithm

```
1. Load best_params.json (current best: 89.389)
2. Perturb 1-3 random parameters (widened after 500+ rejections)
3. Backtest against 8 rounds x 5 seeds (vectorized, ~70ms)
4. Accept if score improves (greedy)
5. Accept with 20% probability if within 0.05 of best (Metropolis)
6. Write best_params.json on improvement
7. Repeat indefinitely
```

Key Algorithms

Hierarchical Calibration Lookup

```
weight_fine = cal_fine_base + count / cal_fine_divisor  (capped at cal_fine_max)
weight_coarse = cal_coarse_base + count / cal_coarse_divisor
weight_base = cal_base_base + count / cal_base_divisor

prior = weighted_average(fine, coarse, base, global)
```

Score Blending (empirical FK + calibration)

```
blended = prior_weight * calibration + emp_strength * empirical
blended /= sum(blended)  # renormalize
```

Global Multiplier

```
ratio = observed_count[class] / expected_count[class]
dampened = ratio ^ power[class]
clamped = clip(dampened, class_min, class_max)
prediction *= clamped  # per class
```

File Map

File	Purpose
`daemon.py`	Round monitor, exploration trigger, submission loop
`explore.py`	Viewport selection, observation collection
`predict_gemini.py`	Production prediction engine
`fast_predict.py`	Vectorized prediction (numpy, for autoloop)
`calibration.py`	Hierarchical prior model
`sim_inference.py`	CMA-ES simulator fitting
`sim_model_gpu.py`	PyTorch GPU simulator
`autoloop_fast.py`	Continuous parameter optimization
`utils.py`	GlobalMultipliers, FeatureKeyBuckets, ObservationAccumulator
`client.py`	API client with rate limiting
`config.py`	Constants (MAP_W=40, NUM_CLASSES=6, PROB_FLOOR=0.01)
`best_params.json`	Current best parameters (auto-updated by autoloop)

NorgesGruppen Object Detection

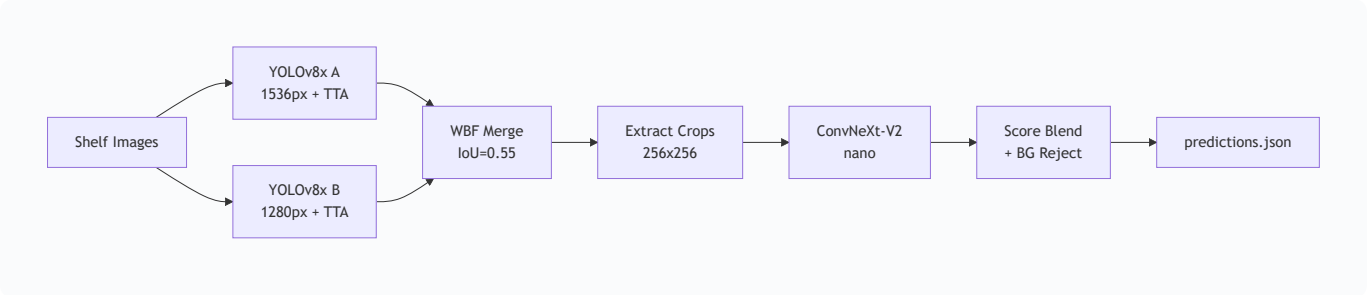
Detect and classify grocery products on store shelf images using a two-stage ensemble pipeline: multi-model YOLO detection fused with Weighted Boxes Fusion, followed by ConvNeXt-V2 re-classification.

Features

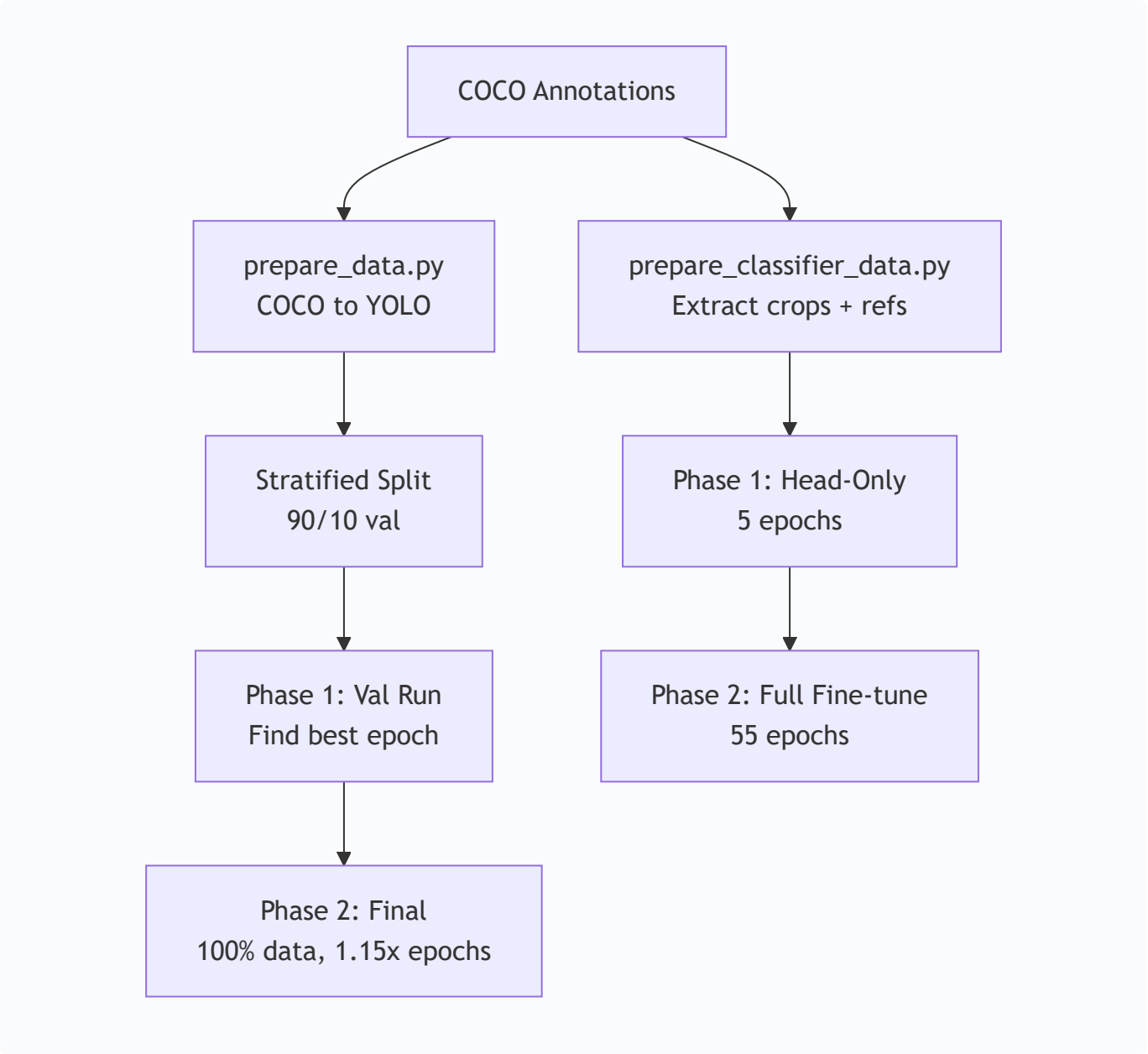
- Dual YOLOv8x detection with test-time augmentation (TTA)
- Weighted Boxes Fusion (WBF) for multi-model ensemble merging
- ConvNeXt-V2 nano re-classification with smart score blending
- Background rejection via trained class 356
- Rare category protection through stratified data splitting
- Sandbox-compatible inference (blocked imports, torch.load patch)
- 285s safety timeout (300s sandbox limit)

User Flows

Inference Pipeline



Training Pipeline



Acceptance Criteria

- ☑ Detection mAP@0.5 > 0.95 on train set
- ☑ Classification mAP@0.5 > 0.95 on train set
- ☑ Combined score > 0.97 on train set
- ☑ Runs within 300s on L4 GPU
- ☑ Zip under 420 MB
- ☑ No blocked imports
- ☑ torch.load monkeypatch applied before ultralytics import

- [ ] Test set combined score > 0.95

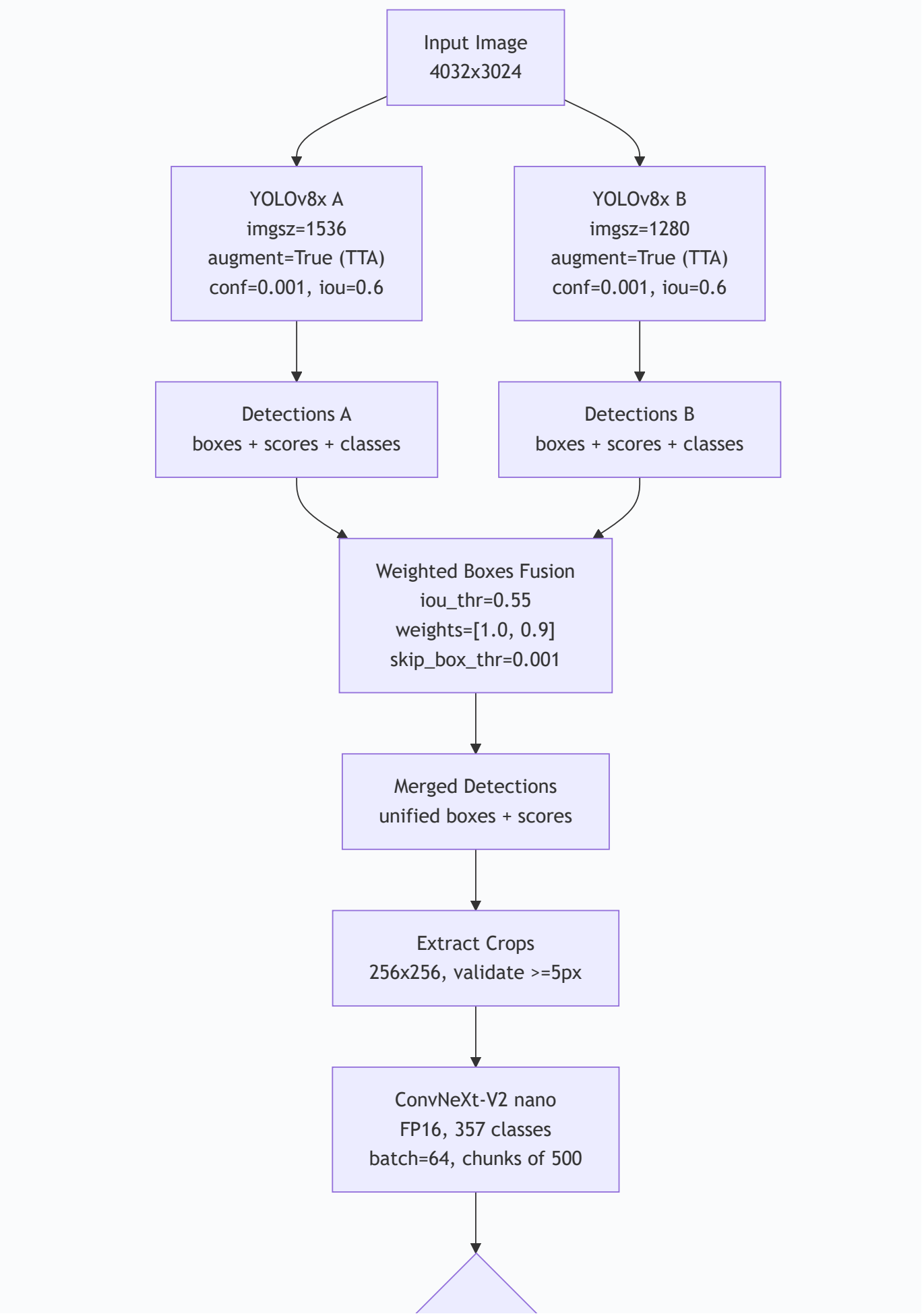
Edge Cases

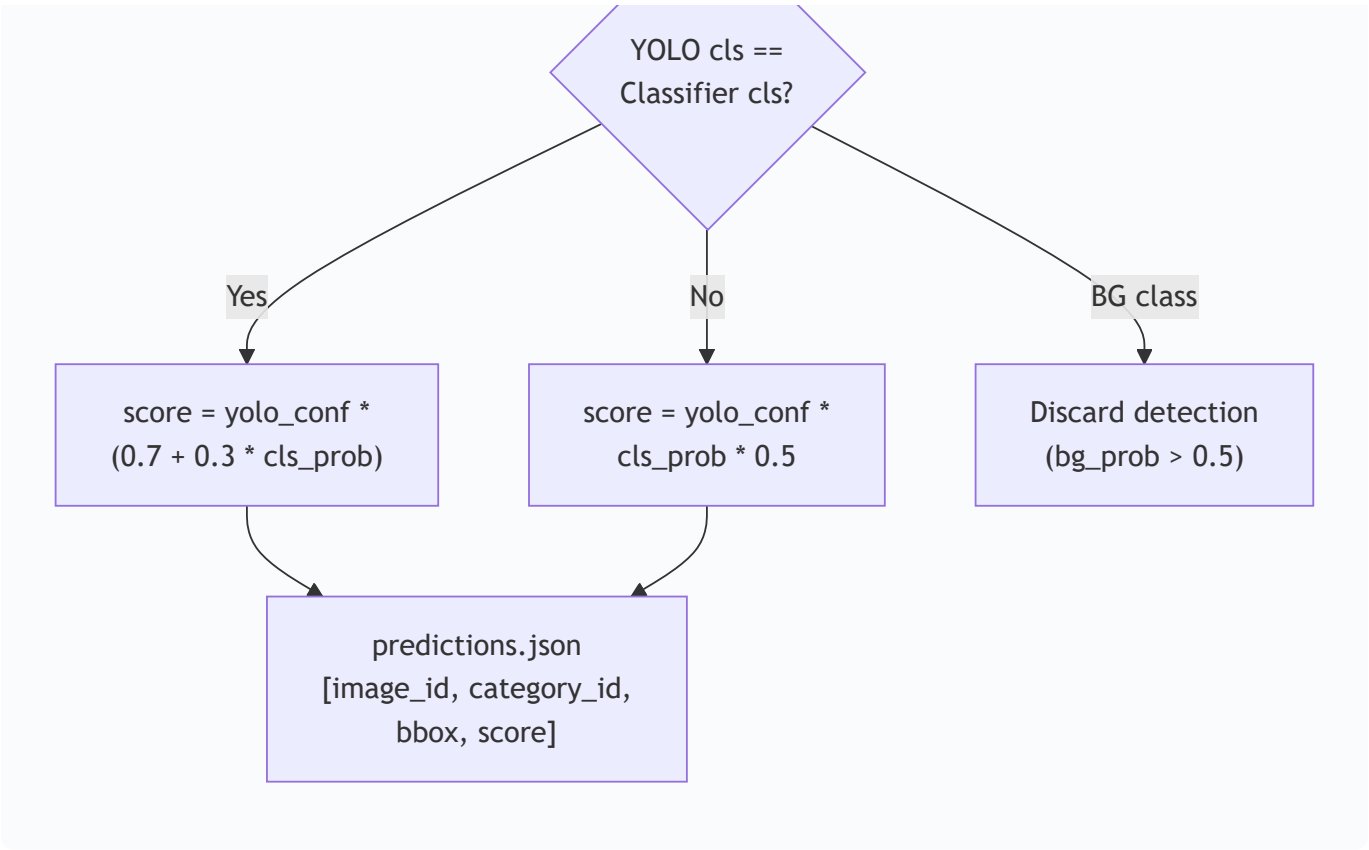
- Products with identical packaging but different sizes (e.g., 500g vs 1kg)
- Occluded products where only partial label is visible
- Empty shelf regions triggering false positive detections (mitigated by BG class)
- Rare categories with 1-3 training examples (protected by stratified split)
- TTA timeout risk on large images (mitigated by 285s safety cutoff)

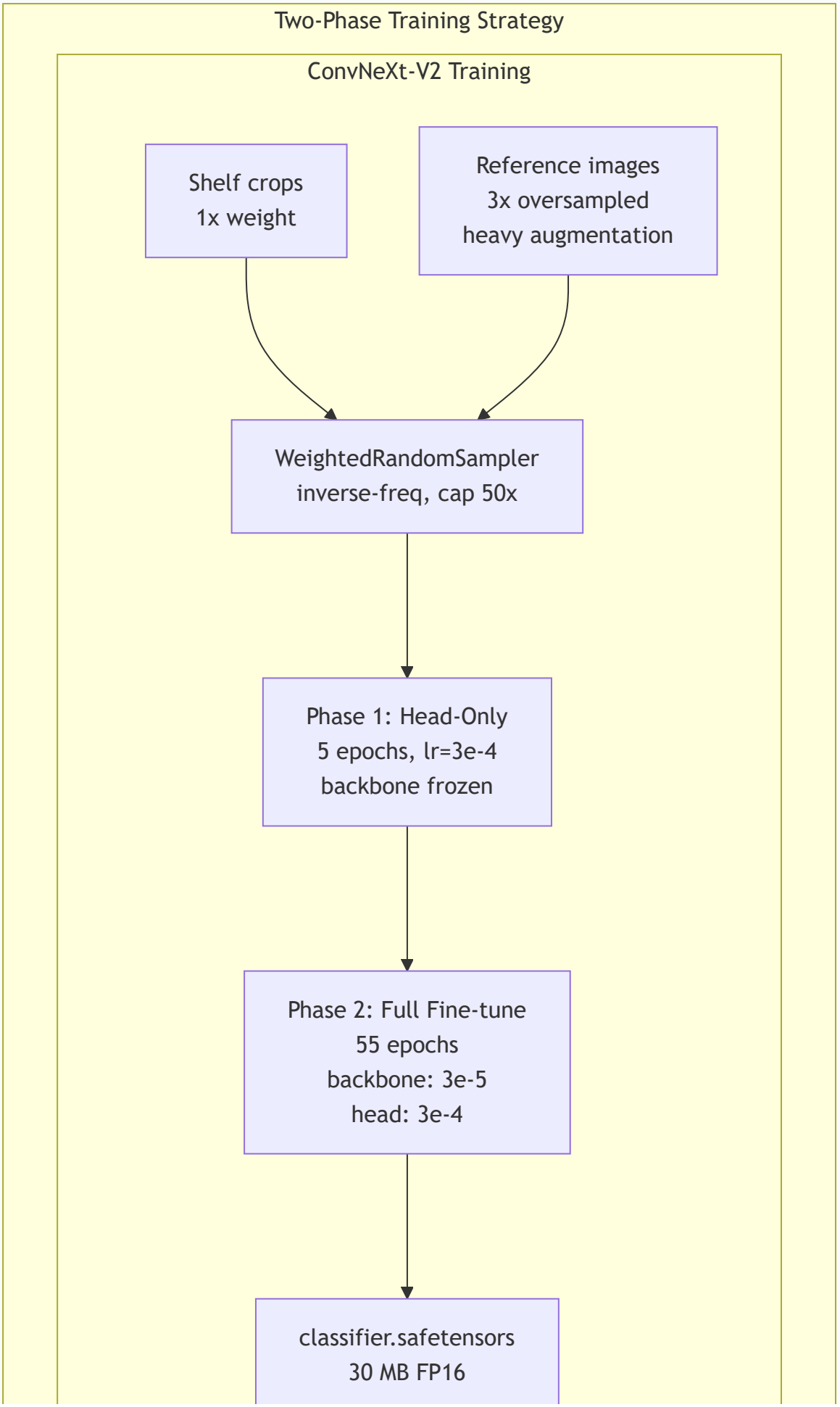
NorgesGruppen Detection - Technical Design

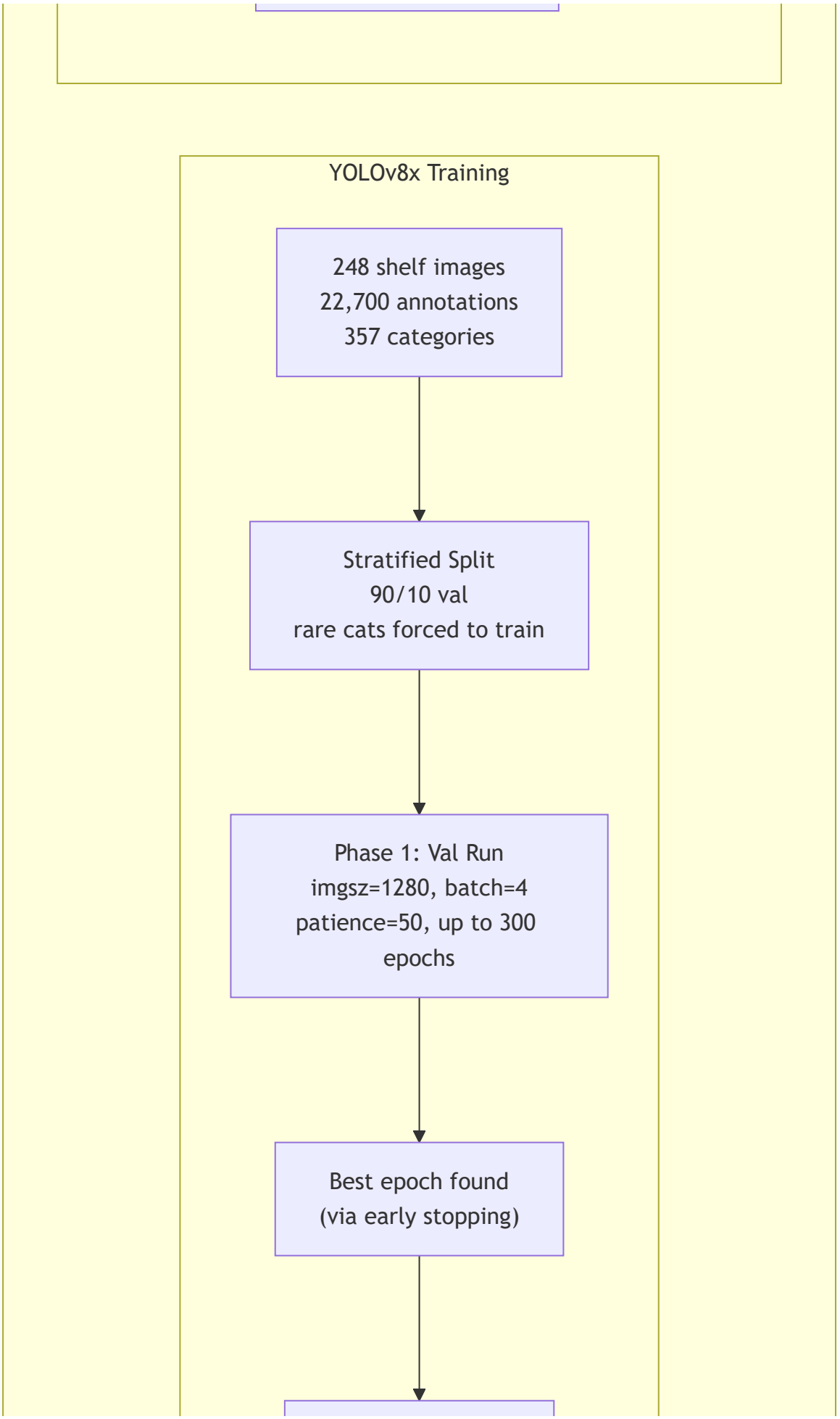
Two-stage detection + classification pipeline optimized for grocery shelf product recognition within a constrained Docker sandbox.

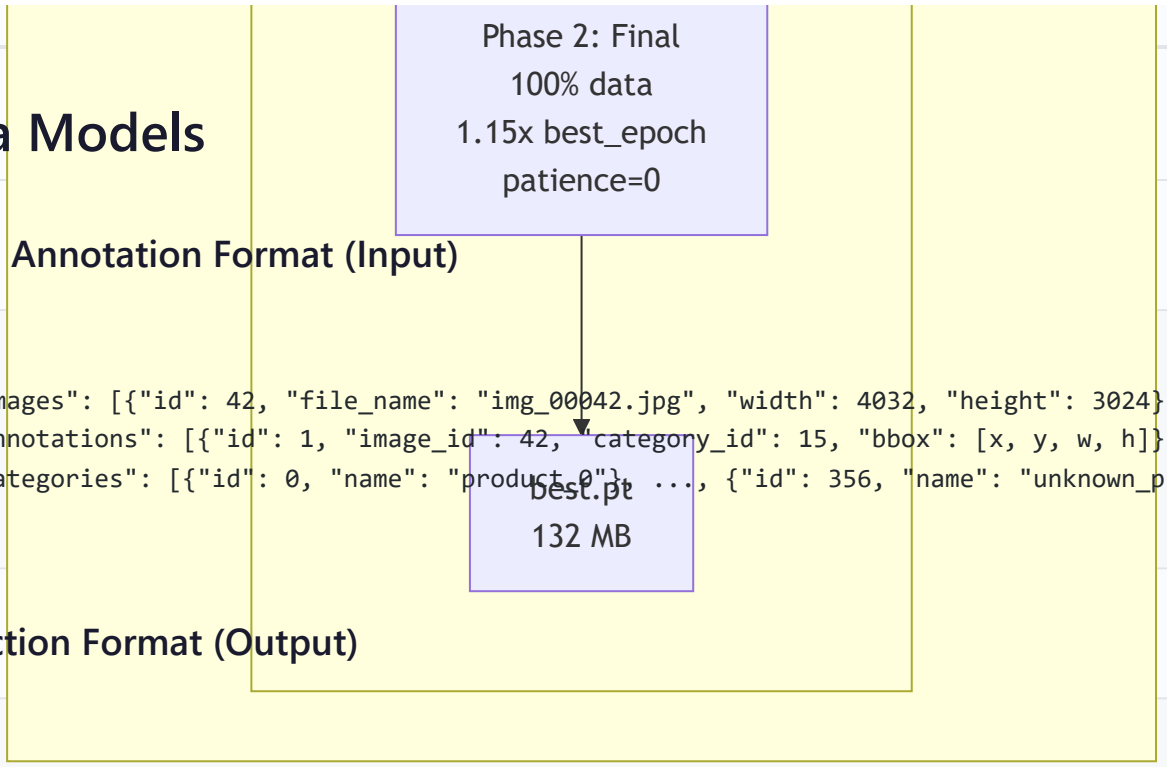
Inference Pipeline Detail


Input Image
4032x3024
YOLOv8x A
imgsz=1536
augment=True (TTA)
conf=0.001, iou=0.6
YOLOv8x B
imgsz=1280
augment=True (TTA)
conf=0.001, iou=0.6
Detections A
boxes + scores + classes
Detections B
boxes + scores + classes
Weighted Boxes Fusion
iou_thr=0.55
weights=[1.0, 0.9]
skip_box_thr=0.001
Merged Detections
unified boxes + scores
Extract Crops
256x256, validate >=5px
ConvNeXt-V2 nano
FP16, 357 classes
batch=64, chunks of 500


YOLO cls == Classifier cls?
Yes
No
BG class
score = yolo_conf * (0.7 + 0.3 * cls_prob)
score = yolo_conf * cls_prob * 0.5
Discard detection (bg_prob > 0.5)
predictions.json [image_id, category_id, bbox, score]


Two-Phase Training Strategy
ConvNeXt-V2 Training
Shelf crops
1x weight
Reference images
3x oversampled
heavy augmentation
WeightedRandomSampler
inverse-freq, cap 50x
Phase 1: Head-Only
5 epochs, lr=3e-4
backbone frozen
Phase 2: Full Fine-tune
55 epochs
backbone: 3e-5
head: 3e-4
classifier.safetensors
30 MB FP16


YOLOv8x Training
248 shelf images
22,700 annotations
357 categories
Stratified Split
90/10 val
rare cats forced to train
Phase 1: Val Run
imgsz=1280, batch=4
patience=50, up to 300
epochs
Best epoch found
(via early stopping)



Data Models

COCO Annotation Format (Input)

```
{
  "images": [{"id": 42, "file_name": "img_00042.jpg", "width": 4032, "height": 3024}],
  "annotations": [{"id": 1, "image_id": 42, "category_id": 15, "bbox": [x, y, w, h]}],
  "categories": [{"id": 0, "name": "product_0"}, ..., {"id": 356, "name": "unknown_product"
}
```

Prediction Format (Output)

```
[
  {"image_id": 42, "category_id": 15, "bbox": [x, y, w, h], "score": 0.87},
  ...
]
```

Training Data

- 248 shelf images, ~22,700 annotations, 357 categories
- 327 product reference images with multi-angle photos
- Stratified split: rare categories (<=3 annotations) forced to training set

---

Inference Pipeline

Stage 1: Detection (YOLO)

Model	Weight File	Input Size	TTA	Weight
YOLO-A	`best_a.pt`	1536px	Yes	1.0
YOLO-B	`best_b.pt`	1280px	Yes	0.9

Parameters: `conf=0.001`, `iou=0.6`, `max_det=3000`

Stage 2: WBF Fusion

```
from ensemble_boxes import weighted_boxes_fusion
# Normalize boxes to [0,1], merge, denormalize
# iou_thr=0.55, skip_box_thr=0.001
```

Stage 3: Classification (ConvNeXt-V2)

- Model: ConvNeXt-V2 nano (15.2M params, 30 MB FP16 safetensors)
- Input: 256x256 crops extracted from detection boxes
- Output: 357 class probabilities
- Batch size: 64 crops, chunked at 500 crops

Stage 4: Score Blending

```
if classifier_class == yolo_class:
    score = yolo_conf * (0.7 + 0.3 * cls_prob)    # agreement boost
elif classifier_class == BG_CLASS:
    score = 0  # reject detection (bg_prob > 0.5)
else:
    score = yolo_conf * cls_prob * 0.5              # disagreement penalty
```

Training Architecture

YOLOv8x Training (train_yolo.py)

Phase 1 -- Validation Run (90/10 split)

- `imgsz=1280`, `batch=4`, `epochs=300`, `patience=50`
- AdamW optimizer, lr=0.001, cosine annealing to 0.00001
- Mosaic=1.0, mixup=0.0, copy_paste=0.1
- Output: best epoch count via early stopping

Phase 2 -- Final Training (100% data)

- Same hyperparameters, epochs = 1.15 * best_epoch
- No early stopping (`patience=0`)
- Output: `best.pt` trained on all data

ConvNeXt-V2 Classifier Training (train_classifier.py)

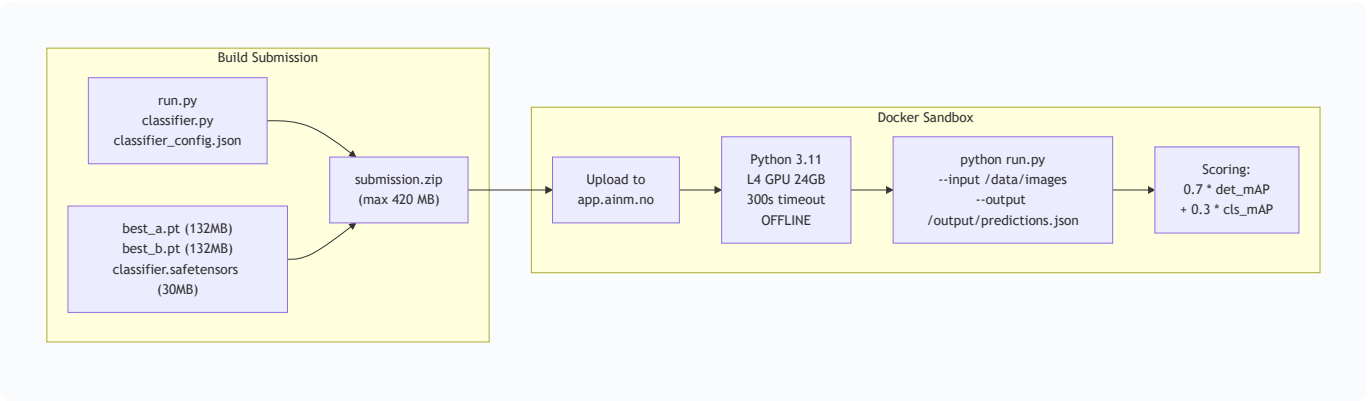
Phase 1 -- Head-Only (5 epochs)

- Freeze backbone, train classification head only
- lr=3e-4

Phase 2 -- Full Fine-tune (55 epochs)

- Unfreeze all, differential learning rates (backbone: 3e-5, head: 3e-4)
- CosineAnnealingLR, label smoothing=0.1
- WeightedRandomSampler (inverse-frequency, capped at 50x)
- Reference images 3x oversampled (bridge white-bg to shelf-bg domain gap)

Submission Pipeline



Sandbox Compatibility

Critical: torch.load Monkeypatch

```
# MUST be before `from ultralytics import YOLO`
_original_torch_load = torch.load
def _patched_torch_load(*args, **kwargs):
    if "weights_only" not in kwargs:
        kwargs["weights_only"] = False
    return _original_torch_load(*args, **kwargs)
torch.load = _patched_torch_load
```

Package Version Pinning

Package	Sandbox Version	Breaking Change
ultralytics	8.1.0	8.2+ weights fail to load
timm	0.9.12	1.0+ weights fail to load
PyTorch	2.6.0+cu124	weights_only=True default
onnxruntime	1.20.0	opset > 20 fails

Submission Structure

```
submission.zip (max 420 MB)
├── run.py              # Entry point
├── classifier.py       # Model loader
├── classifier_config.json
├── classifier.safetensors  # 30 MB
├── best_a.pt               # 132 MB
└── best_b.pt               # 132 MB (optional)
```

Timing Budget (~285s safety margin)

Stage	Per Image	Total (248 imgs)
Model loading	--	10-15s
YOLO-A (1536 + TTA)	~1.0s	~248s
YOLO-B (1280 + TTA)	~0.8s	~198s
WBF merge	<0.01s	<2s
Classifier (batch 64)	~0.05s/crop	~15s
Total		**~270s**

File Map

File	Purpose
`run.py`	Submission entry point (detection + classification)
`classifier.py`	ConvNeXt-V2 model loader (timm + safetensors)
`train_yolo.py`	Two-phase YOLOv8x training
`train_classifier.py`	Two-phase ConvNeXt-V2 training
`prepare_data.py`	COCO to YOLO format + stratified split
`prepare_classifier_data.py`	Crop extraction + reference image organization
`evaluate.py`	COCO evaluation (det mAP + cls mAP)
`sweep.py`	Hyperparameter sweep testing
`synth_test.py`	Local synthetic evaluation

Tripletex AI Accounting Agent

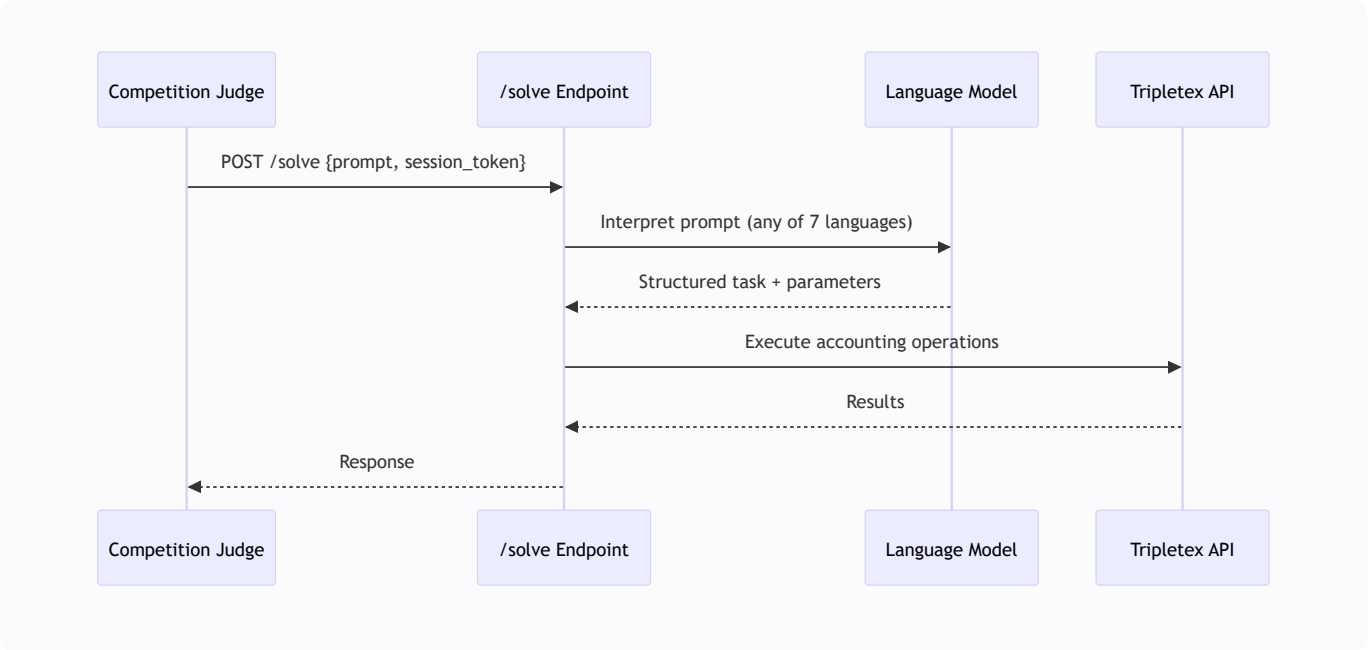
HTTPS endpoint that receives accounting task prompts in 7 languages and executes them via the Tripletex API.

Features

- HTTPS `/solve` endpoint accepting task prompts
- Multi-language prompt interpretation (nb, en, es, pt, nn, de, fr)
- 30 task types across 7 categories: employees, customers, products, invoicing, travel expenses, projects, departments
- 56 variants per task type (1,680 total scenarios)
- Tiered scoring with efficiency bonuses

User Flows

Task Execution



Scoring

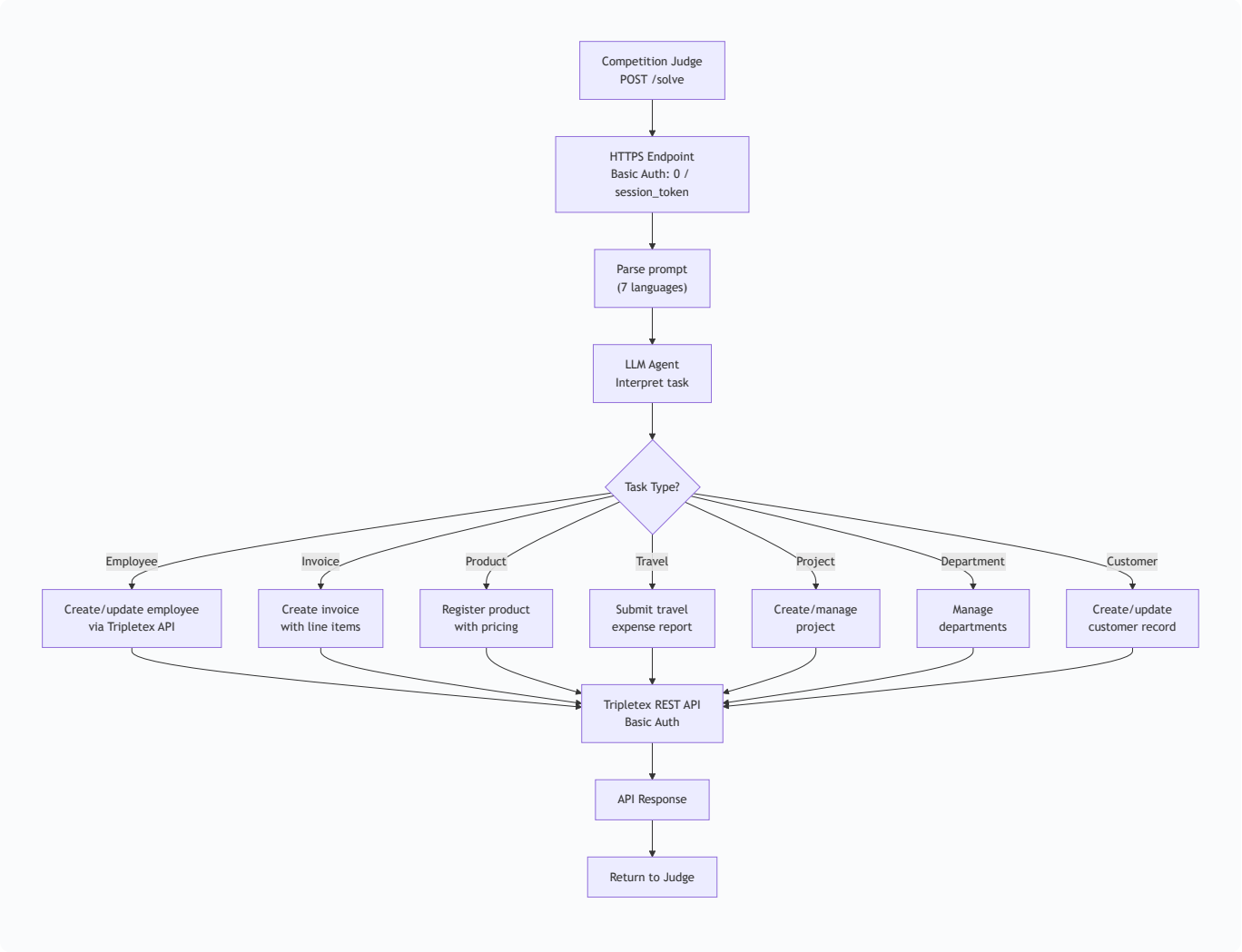
```
score_per_task = correctness * tier_multiplier + efficiency_bonus
max_per_task = 6.0
```

- **Correctness:** Binary (task completed correctly or not)
- **Tier multiplier:** Higher for harder task types
- **Efficiency bonus:** Fewer API calls = higher bonus

Acceptance Criteria

- [ ] HTTPS endpoint deployed and accessible
- [ ] Basic Auth: username `0`, password = session token
- [ ] Handles all 30 task types
- [ ] Handles all 7 input languages
- [ ] Response within timeout limits
- [ ] Correct Tripletex API usage per task type

Intended Architecture



Status

Not started. Requires:

1. HTTPS endpoint deployment (not a zip submission like NorgesGruppen)
2. LLM integration for prompt interpretation
3. Tripletex API client for task execution
4. Task type routing and parameter extraction

Documentation available at `docs/tripletex/` (overview, endpoint, scoring, examples, sandbox).

Development Workflow

Daily development loop for each challenge.

Astar Island

Improving Predictions

1. **Check autoloop progress** -- review `best_params.json` score and experiment count
2. **Analyze weak rounds** -- identify boom vs non-boom score gaps
3. **Propose structural changes** -- modify `predict_gemini.py` or `calibration.py`
4. **Backtest locally** -- run against calibration data (17 rounds x 5 seeds)
5. **If improvement:** update autoloop parameter space if new knobs added
6. **Wait for next round** -- daemon auto-submits with latest params

Adding New Features

1. Add parameter to `best_params.json` schema
2. Implement in `predict_gemini.py` (production) and `fast_predict.py` (vectorized)
3. Add to autoloop perturbation space in `autoloop_fast.py`
4. Backtest shows improvement before committing

Monitoring Active Rounds

The daemon handles everything automatically. To intervene:

- Check `data/daemon.log` for submission status
- Use web dashboard for real-time scores
- Manual submission: `python submit.py` (for custom strategies)

NorgesGruppen

Improving Scores

1. **Evaluate current submission** -- `python evaluate.py` for train set metrics

2. **Identify weakness** -- detection mAP vs classification mAP
3. **Train improved model** -- one GPU job at a time
4. **Test locally** -- `python synth_test.py`
5. **Build submission** -- zip with required files
6. **Submit** -- upload at `app.ainm.no` (max 3/day)

Pre-submission Checklist

- [ ] `run.py` starts with torch.load monkeypatch
- [ ] No blocked imports (`os` , `sys` , `subprocess` , etc.)
- [ ] Zip under 420 MB
- [ ] Max 3 weight files, max 10 Python files
- [ ] Weight files trained with `ultralytics==8.1.0` and `timm==0.9.12`
- [ ] Tested locally: completes in <285s

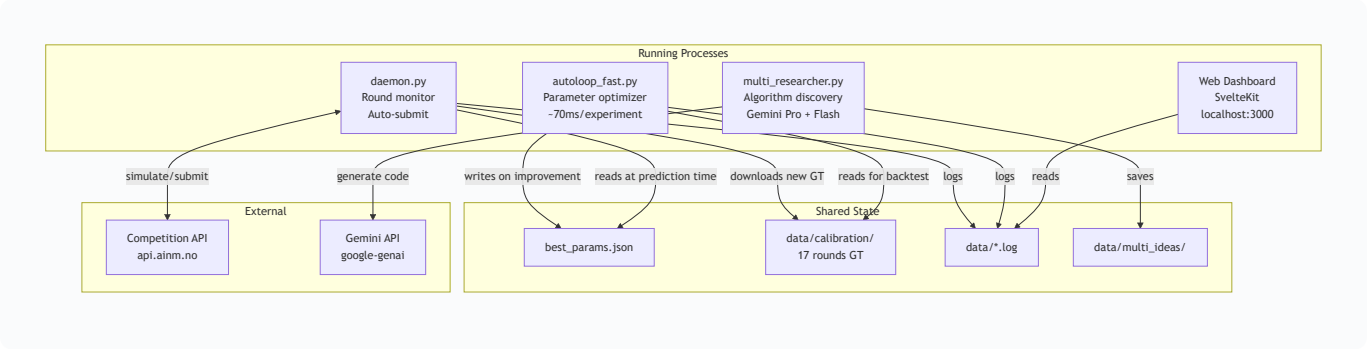
General Rules

- **Never run concurrent GPU training** -- sequential only, CUDA OOM risk
- **Pin package versions** -- match sandbox exactly
- **Save ideas to IDEAS.md** -- document findings and hypotheses
- **Use `-u` flag** -- all Python daemon/autoloop processes need unbuffered output

Operations Guide

How to start, monitor, and manage all running systems.

System Process Map



Astar Island -- Full System Startup

All Python processes require the `-u` flag for unbuffered output (real-time logging).

1. Daemon (round monitor + auto-submission)

```
cd astar-island-solution
python -u daemon.py 2>&1 | tee data/daemon.log
```

Monitors API every 60s. On new round: explores, predicts, submits, then iterates.

2. Autoloop (parameter optimization)

```
cd astar-island-solution
python -u autoloop_fast.py 2>&1 | tee data/autoloop_fast_output.log
```

Runs continuously. Updates `best_params.json` on improvements. ~70ms per experiment.

3. Multi-Researcher (structural improvements)

```
cd astar-island-solution
python -u <researcher_script>.py 2>&1 | tee data/multi_researcher_output.log
```

Generates algorithm variants in `data/multi_ideas/`. Requires API keys in `.env`.

4. Web Dashboard

```
cd astar-island-solution/web
npm run dev
```

Opens at `http://localhost:3000`. Connects to API at `localhost:7091`.

NorgesGruppen -- Training & Submission

Training YOLOv8x

```
cd norgesgruppen-solution
python train_yolo.py
```

Two-phase: validation split (finds best epoch) then full data training. Takes ~4-6 hours on GPU. **Never run concurrent GPU training jobs.**

Training Classifier

```
cd norgesgruppen-solution
python train_classifier.py
```

Two-phase: head-only warmup then full fine-tune. Takes ~2-3 hours.

Building Submission

```
cd norgesgruppen-solution
zip -r ../submission.zip . -x ".*" "__MACOSX/*" "datasets/*" "classifier_data/*" "runs/*"
```

Verify zip is under 420 MB and contains `run.py` + weight files.

Local Evaluation

```
cd norgesgruppen-solution
python evaluate.py
python synth_test.py  # synthetic test
```

Environment Variables

API keys and tokens stored in `.env` file (not committed). Required for:

- Astar Island API authentication
- Gemini API (researcher agents)
- Any other external service calls

Monitoring

Astar Island Scores

- Web dashboard: `http://localhost:3000` (scores, autoloop, daemon status)
- Autoloop log: `data/autoloop_fast_output.log` (experiment results)
- Daemon log: `data/daemon.log` (round detection, submissions)
- Best params: `best_params.json` (current optimal, auto-updated)

NorgesGruppen Scores

- Competition leaderboard: `https://app.ainm.no`
- Local eval: `python evaluate.py` (train set mAP)

Troubleshooting

Issue	Cause	Fix
CUDA OOM	Concurrent GPU jobs	Kill other GPU processes first
ultralytics load error	Wrong version weights	Retrain with `ultralytics==8.1.0`
torch.load fails	Missing monkeypatch	Add patch before `from ultralytics import YOLO`
Rate limit 429	Too many API calls	Daemon handles with exponential backoff
Submission rejected	Blocked import detected	Replace with allowed alternative (pathlib, json)
Autoloop stagnant	Local optimum	Will auto-widen perturbation after 500 rejections